Machine Translation for English $\Longleftrightarrow$ Indian Languages

Comprehensive Report

Saumitra Yadav
Roll No.- 201507647
Ph.D. CSE
Language Technologies Research Center,
KCIS, IIIT-Hyderabad

Contents

1 Introduction **1**

2 Machine Translation Architectures and Their extensions **5**
2.1 Statistical Models . . . 5
2.1.1 Phrase Based Machine Translation System . . . 5
2.1.2 Hierarchical Phrase Based Machine Translation Model . . . 8
2.1.3 Factor Based Statistical Model . . . 9
2.2 Neural Models . . . 10
2.2.1 Word Representation in NMT . . . 10
2.2.2 Formulation of Neural Machine Translation . . . 11
2.2.3 Factored Neural Machine Translation . . . 14
2.2.4 Copy Attention For NMT . . . 14
2.2.5 Coverage Model For NMT . . . 15
2.2.6 Context Gate for NMT . . . 17
2.2.7 Reconstruction of Source in NMT . . . 18
2.2.8 Transformer . . . 18
2.2.9 Document Level Context in NMT . . . 19
2.3 Evaluation of MT systems . . . 20
2.4 For Indian Languages . . . 20
2.5 Our Experiments . . . 21

3 Preprocessing **25**
3.1 Subwords . . . 25
3.2 Reordering . . . 27
3.3 Segment and Translate . . . 28
3.4 Our Experiments . . . 29

4 Methods For Low resource Language Pair **35**
4.1 Pivot Translation . . . 35
4.1.1 Pivoting in SMT . . . 35
4.1.2 Pivoting in NMT . . . 36
4.2 Data Augmentation . . . 36
4.3 Transfer learning . . . 38
4.4 Multilingual Neural Machine Translation . . . 39
4.5 Unsupervised Machine Translation . . . 40

5 Going Forward **42**

6 List of Selected Papers **53**

List of Figures

1.1 Architecture of Sampark . . . 3

2.1 Algorithm to find lexical translation and alignment probability. . . . 6
2.2 An example of sentence pair matrix . . . 6
2.3 List of all possible extracted phrases . . . 7
2.4 Example of Factored Model . . . 9
2.5 Example of 1 hot encoding . . . 10
2.6 Word2vec methods . . . 11
2.7 LSTM encoder processed input sentence and produce tokens on target side in auto-regressive manner using LSTM decoder. . . . 12
2.8 Step by Step demonstration of how to find scores with various source tokens and word generated in previous time step, to find the attention weights for source tokens embeddings. . . . 13
2.9 Factored NMT with usage of various features embedded together with word embedding of source token. . . . 14
2.10 Copy Attention Model for either generating the target token or copying the source token. . . . 15
2.11 Coverage model to aggregate attention after each time step so that it can be used in translation for future steps. . . . 16
2.12 Show attention weights in green at the top and resultant coverage for each token below their respective position in graph. . . . 16
2.13 Context Gate to use in identifying relative importance of previous target word generated and context to generate current word. . . . 17
2.14 Using forward and backward translation to improve translation learning. . . . 18
2.15 Using self-attention to encode importance of other source word for a word and decoding to generate translation. . . . 18
2.16 Using Document level Context by using attention mechanism in hierarchical fashion. 19
2.17 Average Fluency and Adequacy score for each translation systems with respect to input sentence length. . . . 24

3.1 Iterative algorithm to find the subword using BPE, most frequent bigram at each merge is underlined. . . . 26
3.2 Bracketing transduction grammar . . . 27
3.3 Pre-ordering For English Hindi translation. To keep diagram uncluttered, some words following same order after pre-ordering have been grouped together and marked as underlined. . . . 28
3.4 BTG tree representing the pre-ordering . . . 28
3.5 Average Fluency and Adequacy score for each translation systems with respect to input sentence length. . . . 33

4.1 Flow of system training for data augmentation using Back translation . . . 37
4.2 Example of transfer learning. . . . 39
4.3 Toy illustration of the three principles of unsupervised MT . . . 41

List of Tables

2.1 Weights on reordering model for English → Marathi and Reordered English → Marathi systems . . . 8
2.2 BLEU Scores for MT systems using IIT B dataset and respective MT architectures. 21
2.3 Translations for Sample 1 . . . 22
2.4 Translations for Sample 2 . . . 22
2.5 Translations for Sample 3 . . . 23
2.6 NE Coverage for each system . . . 24

3.1 Different Preprocess on Text . . . 25
3.2 BLEU Scores for English To Hindi Translation systems . . . 30
3.3 Translation Output of Example 1 after preprocessing . . . 30
3.4 Translation Output of Example 2 after preprocessing . . . 31
3.5 Translation Output of Example 3 after preprocessing . . . 32
3.6 NE Coverage for each system from Section 2.5 . . . 33
3.7 NE coverage for Each system after preprocessing. . . . 33

4.1 Different ways of generating synthetic dataset . . . 38

Chapter 1

Introduction

There is a saying for Languages in India 'कोस कोस पर बदले पानी और चार कोस पर वाणी' (literal translation: water changes every 3 KM and language every 12). Now this may not be true in literal sense for every part of the country but it gives us a glimpse of language diversity in India. This diversity is visible in scripts we use to communicate and is even more prevalent in spoken medium.

According to Census of 2011[1], India has 22 scheduled and 99 unscheduled languages. If we combine both of these categories, there are at least 62 mother tongues which are spoken by more than 10 million people. And these 62 are just a fraction of total number of languages spoken in India. This gives us the scale of languages available in the country.

If one look at only scheduled languages, one would see there are similarities and differences all across the board. If you look at languages like Hindi and Urdu you can see that there is sharing of vocabulary to some extent. This is due to the shared history between these languages. Similarly, in terms of scripts used, there is similarity in Hindi, Marathi and Konkani. In all the languages, there is number agreement associated with Noun or pronouns. But we can also observe the divergence among languages. For example, degree of agglutination differs in Hindi, Marathi and Telugu. There can also be change in how clauses are embedded, case in point - relative clause in Hindi and Telugu. Another example of divergence is use of classifiers accompanying noun in languages like Bengali, Assamese, Maithili and Nepali. This is atypical for Indo-Aryan language.

Then we have English as one of the Official languages for Union of India. This has different syntax compared to other Indian languages. Also, one needs to just look at information in digital space to see the position of importance English has held across globe. There is information (stories, news, judiciary, educational content) in various Indian languages as well as in English. This can be ascribed to a particular culture, history and place. Due to increase in consumption of Indian languages on digital format, advent of internet and English being a major means of communication across globe; it is important for systems to be there which can transfer information to and from English language to Indian languages.

This entails the use of machine translation systems for transfer of information from one language (will be termed as source language throughout this report) to another (likewise will be termed as target language). A broad definition of machine translation is modeling a system which takes in an input in one language and translates it into another language, while being faithful to source sentence and fluent in target, by capturing the differences and similarities between two languages.

Aforementioned systems can be built using the following methods,

- Rule based systems: As the name suggests, rule-based systems depend on deriving rules to analyse a sentence, find all its features (E.g., Parts of speech of constituent words, finding syntax etc.) and then transfer this knowledge to generator to get the sentence in another language (E.g., Systran[2]). Rule based systems will work well but they suffer from the following major drawbacks,

[1] http://www.censusindia.gov.in/2011Census/Language-2011/Statement-1.pdf
[2] Systran was rule based till early 2000's.

  - Divergence between two ever expanding languages is too volatile to take care of.
  - Difficult to identify and apply each and every rule possible while translating between two languages and thus resultant system built can be fragile when an unknown type of sentence comes as an input.

- Data driven systems: system make use of parallel[3] and monolingual text to learn translation and all other modules associated with it.

  - Statistical systems: [42], [10], [40] these systems try to achieve translation by observing the distribution of correlated words across parallel text.
  - Neural network systems: [89], [6], [97] these systems try to learn translation rules as latent variables and generate target sentences from that, while projecting input tokens as points in an embedding space.

  Both data driven systems can adapt well to learn translation for a specific domain given enough data. Quality and quantity of data dictate the performance of these systems. Although for these two (Statistical and Neural network based) systems, introduction of new datasets can alleviate the problem to some extent.

- Hybrid MT systems: (E.g. Sampark [5][4]) these systems work on combining and using models from systems build using different paradigm to produce translation.

Another way of classifying how translation systems can be built is by how parts of them are trained. We have seen two kinds of systems for translation,

- Modular system: Here translation is divided into multiple steps. And there are sub-model(s) for each step. To create a full system, these sub models are combined. Some of the examples of modular systems are as follows,

  - Sampark [5] is a modular system. It makes use of analysis-transfer-generate paradigm. Here translation is done in phase wise manner (Figure 1.1) by first analysing the sentence to obtain its linguistic features. In transfer phase, model does a lexical translation of lemma and transfer of constituent tree. In generate phase system uses lemma and other features model to generate sentences.
  - Statistical model for translation is another example for modular system. Here, the model learns lexical translation, fluency of target language and reordering of source to match target word order separately via different sub-models. Then the model learns how to use all these sub-models in conjunction to produce translation.

  In this system one can track the origin of an error if any. However, this system requires that each sub-model works well for model to produce good translation.

- Joint system: Neural models [89], [6] are good examples for Joint system. In this, model learns all latent aspects of translation *jointly* to generate translation such that there is no need for separate training for different sub-models. But there is difficulty in pinpointing the origin of error if any in translation output.

Each of the models preludes what are the resource requirements for them, to build translation systems. For Indian Language context, there are some languages like Hindi, Urdu, Marathi with good linguistic resources. But coverage of resources is not uniform across the board. There are parallel datasets (Samantar[5] [76], wat2020[6], CVIT PIB[7], OPUS[8] [92]) for various Indian Languages $\Longleftrightarrow$ English with varying degree of size. E.g. In samantar [76], English Hindi has decent amount of parallel text (8.29 million bitext) when compared with English Odia (0.98 million bitext). And there are data-driven techniques existing, which have attempted to make use of parallel text with

[3] Parallel text or bitext is collection of text along with its translation in another language.
[4] http://sampark.iiit.ac.in/
[5] https://indicnlp.ai4bharat.org/samanantar/
[6] http://lotus.kuee.kyoto-u.ac.jp/WAT/indic-multilingual/
[7] http://preon.iiit.ac.in/~jerin/bhasha/
[8] https://opus.nlpl.eu/

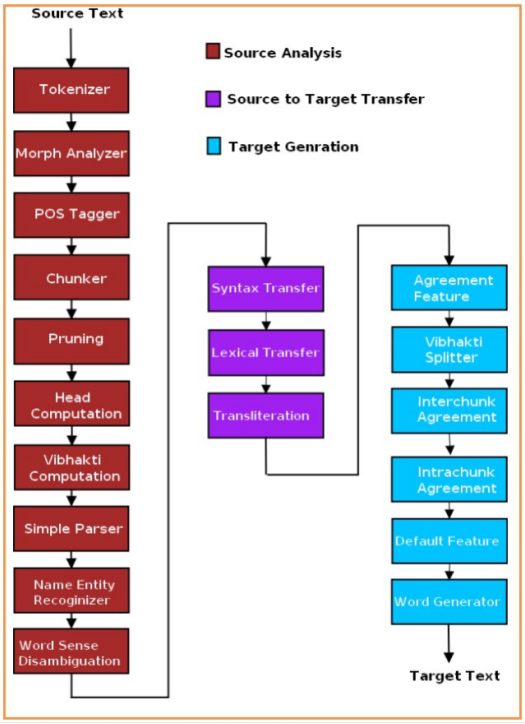


Figure 1.1: Architecture of Sampark

varying degree of translation performance. In this work, we will look at these data-driven methods for building translation systems for language pairs with decent amount of dataset, while also looking at techniques employed to build better translation systems for low resource language pair[9]. Some of the techniques which has been used or can be used to build MT systems for Indian Languages are given below,

1. Using analysis-transfer-generate paradigm to build a modular translation system [5], [21].
2. Using linguistic features with data driven systems [40], [14] to aid MT in generating correct surface form. This entails there are linguistic resources available to be used.
3. Breaking words into smaller units to address data sparsity, agglutination and handling Out of Vocabulary[10] (OOV) tokens [84], [46].
4. Reordering English to match Indian language syntax [67], [47].
5. Pivot based translation systems [57], [48] are systems in which to translate from language **L1** to **L3** we use a pivot language **L2**, such that (**L1**,**L2**) and (**L2**,**L3**) language pairs have good resources.
6. Using data augmentation techniques to create synthetic data which can be added to training data to train translation model [85], [24], [2], [91].
7. Transfer learning [104]: Using High resource bitext to train a MT system, which is later used as starting state for lower resource bitext.
8. Multilingual MT [17], [33]: Making use of multiple language pairs to learn a MT system which can translate from m source languages to n target languages.

In this work as stated earlier, we are exploring literature which has been used to build MT systems for English $\Longleftrightarrow$ Indian languages (from here on referred to as *ENIL setting*). We will

[9]There have not been clear definition as to what is amount of parallel text is required to classify a language pair as low resource, but general consensus is that 10^6 sentence pair can build decent MT system. This is not a strict threshold, as quality and quantity of bitext along with architecture dictate the performance of translation system.
[10]Tokens not present in training dataset

also look at some work which can be used in this context.

In section 2 we will look at various data-driven models used for training MT models. We will also look at how these models are extended to improve translation quality. And we will also look briefly at the question of how MT systems have been evaluated. This section will also discuss how these models have been used for Indian languages. And at the end we will present a brief report on systems we have built based on architectures discussed. For each system we will also present our observations and analysis. After having gone through how MT systems can be built, we will look at what are different types of preprocessing that can be done on datasets used for translation, that is discussed in section 3. This section introduces various methods for preprocessing, ranging from segmenting words using different models to reordering English sentences to match Indian language syntax. We will also look at work done to translate sentences in piece-wise manner. Then, we will look in various preprocessing done for *ENIL setting*. At the end of section, we will look at some of the systems we developed using preprocessing methods mentioned earlier. Then, in section 4 we will look at techniques E.g., Pivoting, Transfer learning, Data augmentation etc. and different ways in which they have been modeled or used successfully for low resource setting.

Section 5 summarises our observations and gives a brief outline of experiments and analyses we will conduct for developing MT systems in *ENIL setting*.

Chapter 2

Machine Translation Architectures and Their extensions

This section deals with data driven models for building machine translation systems. We will first see Statistical models (2.1) and their extensions. Then we will see how MT is modeled as neural network (2.2) and how they have been extended. We will also see our experiments for English $\Longleftrightarrow$ Hindi translation system using some of the models discussed in this section, followed by our analysis for the same.

2.1 Statistical Models

These kinds of models make use of translation probability distribution of words across parallel texts, primarily calculated based on word-based models [8].

2.1.1 Phrase Based Machine Translation System

Statistical machine translation (SMT) is formulated using noisy channel model. Consider that a sentence $\boldsymbol{f}$ from language **L1** is being translated to a sentence $\boldsymbol{e}$ in language **L2**. This is formulated as,

$$\textbf{argmax}_{\mathbf{e}} p(e|f) = \textbf{argmax}_{\mathbf{e}} p(f|e)p(e) \tag{2.1}$$

Here p($f|e$) is lexical translation model. p(e) is language model which takes care that translation generated sentence **e'** is fluent enough to be passed as sentence in language L2. In phrase-based translation (PBSMT) during decoding $\boldsymbol{f}$ is segmented into **I** phrases [42]. Now these phrases are not necessarily linguistic in nature. So the translation model is as follows,

$$p(f|e) = \prod_{i=1}^{I} \phi_i(f_i|e_i) d(a_i - b_{i-1} - 1) \tag{2.2}$$

Where $\phi()$ is lexical translation probability and *d()* is a distortion probability distribution, which measures the probability that phrase $\mathrm{f_i}$ at position i is translated to phrase $\mathrm{e_j}$ at position j. Here $\mathrm{a_i}$ denotes the start position of source phrase that was translated into $\mathrm{i^{th}}$ target phrase and $\mathrm{b_i}$ denotes the end position of previous source phrase which was translated into $\mathrm{i\text{-}1^{th}}$ target phrase.

To get these phrase probabilities, PBSMT starts with word alignments [8]. Figure 2.1 shows algorithm to find the word probability and alignment distribution. *t(e|f)* is lexical translation probability of word e given f. *a(i|j,l_e,l_f)* is the probability that word at position i in source language (**L1**) is translated to word at position j in the target language (**L2**).

Now the same algorithm is applied in opposite direction i.e. considering **L2** as source and **L1** as target language. After getting alignment in both directions Koen et. al. [42] made a sentence pair matrix similar to Figure 2.2.

After getting all the alignment points, a heuristic was used to extract all possible phrase pairs from this matrix. While extracting, these pairs are kept consistent with alignments (phrase pairs with consistent alignments marked in different colours given in Figure 2.2). Phrase pair ($\overline{f}$,$\overline{e}$) is said to be consistent with an alignment A if alignment points of all words in $\overline{f}$(=$\mathrm{f_1}$,$\mathrm{f_2}$,$\mathrm{f_3}$,...) are

```
Input: set of sentence pairs (e,f)
Output: probability distributions t (lexical translation)
        and a (alignment)
 1: initialize t(e|f) uniformly
 2: initialize a(i|j,l_e,l_f) = 1/(l_f+1) for all i,j,l_e,l_f
 3: while not converged do
 4:   // initialize
 5:   count(e|f) = 0 for all e,f
 6:   total(f) = 0 for all f
 7:   count_a(i|j,l_e,l_f) = 0 for all i,j,l_e,l_f
 8:   total_a(j,l_e,l_f) = 0 for all j,l_e,l_f
 9:   for all sentence pairs (e,f) do
10:     l_e = length(e), l_f = length(f)
11:     // compute normalization
12:     for j = 1 .. l_e do // all word positions in e
13:       s-total(e_j) = 0
14:       for i = 0 .. l_f do // all word positions in f
15:         s-total(e_j) += t(e_j|f_i) * a(i|j,l_e,l_f)
16:       end for
17:     end for
18:     // collect counts
19:     for j = 1 .. l_e do // all word positions in e
20:       for i = 0 .. l_f do // all word positions in f
21:         c = t(e_j|f_i) * a(i|j,le,lf) / s-total(e_j)
22:         count(e_j|f_i) += c
23:         total(f_i) += c
24:         count_a(i|j,le,lf) += c
25:         total_a(j,l_e,l_f) += c
26:       end for
27:     end for
28:   end for
29:   // estimate probabilities
30:   t(e|f) = 0 for all e,f
31:   a(i|j,l_e,l_f) = 0 for all i,j,l_e,l_f
32:   for all e,f do
33:     t(e|f) = count(e|f) / total(f)
34:   end for
35:   for all i,j,l_e,l_f do
36:     a(i|j,l_e,l_f) = count_a(i|j,l_e,l_f) / total_a(j,l_e,l_f)
37:   end for
38: end while
```

Figure 2.1: Algorithm to find lexical translation and alignment probability.

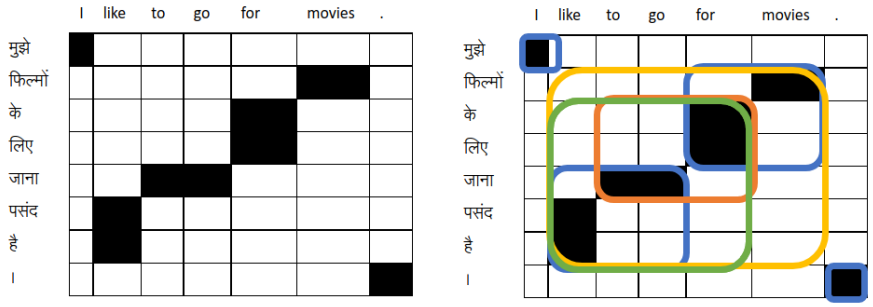


Figure 2.2: An example of sentence pair matrix

aligned with atleast one of the words in $\bar{e}(=e_1,e_2,e_3,...)$ and vice versa, Once these consistent alignments were identified, phrases are extracted using phrase extraction algorithm [37]. Basically, algorithm loops over all possible target phrases and finds the minimal source phrase that matches each of them, while taking following points into account,

- If target phrase contains unaligned phrase, it is not matched it with source phrase
- If source phrase has additional words which are unaligned with target phrase, then this pair is not extracted.
- If source phrase has additional words which are unaligned with any other source phrase, then phrase is extracted.

For figure 2.2 extracted phrases are given in figure 2.3. After getting all possible aligned phrase pairs, a phrase table is created by finding probability of these alignments. Distortion model not only looks at the movement of phrases required, but it also looks at orientation of movement required to rearrange source tokens to get near their respective translation in target language. There are 3 possible orientation models which calculate how translation of source phrase should be moved,

- Monotone: source and target phrase have same order, i.e. in sentence matrix alignments would seem to be covering diagonal cells.
- Swap: source phrase can be swapped with neighbouring phrases to create better translation.
- Discontinuous: position of phrases between source and target is too far.

Now using translation model, reordering model and language model, phrase-based translation

I like to go for movies. - मुझे फिल्मों के लिए जाना पसंद है ।
I - मुझे
. - ।
like - पसंद है
to go - जाना
for - के लिए
movies - फिल्मों
to go for - के लिए जाना
for movies - फिल्मों के लिए
like to go - जाना पसंद है
like to go for - के लिए जाना पसंद है
like to go for movies - फिल्मों के लिए जाना पसंद है

Figure 2.3: List of all possible extracted phrases

model (using equation (2.1) and (2.2)) is:

$$e_{best} = argmax_e \prod_{i=1}^{I} \phi(\bar{f}_i|\bar{e}_i) d(start_i - end_{i-1} - 1) \prod_{i=1}^{|e|} p_{LM}(e_i|e_1 ... ei - 1) \tag{2.3}$$

Equation 2.3 assumes that translation of phrase does not depend on surrounding phrases, this introduced some practical problems while decoding the translation. Hence, in equation 2.3, weights are introduced to all the 3 models to form a log linear model, thus model now becomes

$$e_{best} = argmax_e \prod_{i=1}^{I} \phi(\bar{f}_i|\bar{e}_i)^{\lambda_\phi} d(start_i - end_{i-1} - 1)^{\lambda_d} \prod_{i=1}^{|e|} p_{LM}(e_i|e_1 ... e_{i-1})^{\lambda_{LM}} \tag{2.4}$$

To ensure that we don't see underflow[1] the model is rewritten as a log model and we get,

$$p(e, a|f) = exp \begin{bmatrix} \lambda_\phi \sum_{i=1}^{I} \log \phi(\bar{f}_i|\bar{e}_i) \\ +\lambda_d \sum_{i=1}^{I} \log d(start_i - end_{i-1} - 1) \\ +\lambda_{LM} \sum_{i=1}^{|e|} \log p_{LM}(e_i|e_1 ... e_{i-1}) \end{bmatrix} \tag{2.5}$$

Given a good enough representative data (called tuning or dev or validation data) we can use equation 2.4 with MERT [62] to find good weights. This helps decoder to attune the models based on the nature of two languages with each other. E.g.

- If two languages are Hindi and Punjabi it will be more beneficial for model to have more weightage to phrase table than to language model since both languages have similar syntactic structure.
- For language like English to Marathi, weight in reordering model would be more on swapping or discontinuous orientation then monotonous, due to different in syntax. But say if we reorder English to match Marathi syntax than we will see weight for monotonous orientation increasing (in Table 2.1).

This log linear model also enables the possibility of adding new features (which we will also see in 2.1.3). To consider a new feature, say length ratio of target and source sentence while training we

[1]Situation when absolute value is more precise than the computer can actually represent in memory

	Monotonous E → M	**Swap E → M**	**Discontinuation E → M**	**Monotonous M → E**	**Swap M → E**	**Discontinuation M → E**
Eng to Marathi	0.026	0.022	0.030	0.038	-0.064	0.039
Reordered Eng to Marathi	**0.093 ↑**	**-0.057 ↓**	**-0.045 ↓**	0.0006 ↓	0.080 ↑	0.135 ↑

Table 2.1: Weights on reordering model for English → Marathi and Reordered English → Marathi systems

can add new feature in equation 2.5 as given below,

$$p(e,a|f) = exp\begin{bmatrix} \lambda_\phi \sum_{i=1}^{I} \log \phi(\bar{f}_i|\bar{e}_i) \\ +\lambda_d \sum_{i=1}^{I} \log d(start_i - end_{i-1} - 1) \\ +\lambda_{LM} \sum_{i=1}^{|e|} \log p_{LM}(e_i|e_1...e_{i-1}) \\ +\lambda_{lengthRatio} \left(\frac{TargetLength}{SourceLength}\right) \end{bmatrix} \tag{2.6}$$

Once we have models and their respective weights attuned to language pair, stack decoding [65] is used along with beam search [39] to avoid search space explosion to find the translation of a given input. Now, the following subsection will give some extension done to PBSMT, by remodeling how we decode the source sentence.

2.1.2 Hierarchical Phrase Based Machine Translation Model

In hierarchical Phrase Based Statistical MT (HPBSMT) [10], decoder made use of weighted synchronous CFG [4] to learn rules regarding reordering and translation. Consider synchronous CFG given below,

$$X \rightarrow < \gamma, \alpha, \beta > \tag{2.7}$$

here X is the non-terminal, γ and α are strings of terminals (tokens) and non-terminals, β is one to one correspondence between non-terminals of α and γ. The grammar rules extracted from bitext (mentioned below) use only X, with two exceptions which are termed as 'glue rules' and are used to combine a sequence of Xs to form an S (sentence):

- S $\rightarrow\langle S_1X_2, S_1X_2 \rangle$
- S $\rightarrow\langle \gamma X_1, X_1 \rangle$

These rules give an option to model for building only partial translation using hierarchical phrases and then combine them serially as in a standard phrase-based model. Using log-linear model, weight of each rule is,

$$w(X \rightarrow \langle \gamma, \alpha \rangle) = \prod_i \phi_i(X \rightarrow \langle \gamma, \alpha \rangle)^{\lambda_i} \tag{2.8}$$

where φ_i are features defined on aforementioned rules. These features are as follows,

- $P(\gamma|\alpha)$ and $P(\alpha|\gamma)$ - lexical translation probabilities of both direction
- lexical weights $P_w(\gamma|\alpha)$ and$P_w(\alpha|\gamma)$ [38]
- a phrase penalty exp(1), which allows the model to learn a preference for longer or shorter derivations, analogous to Koehn's phrase penalty [38].

with glue rules being exception to these, 1st glue rule has weight 1 and second one has

$$w(S \rightarrow \langle S_1X_2, S_1X_2) = exp(-\lambda_g),$$

where λ_g controls model's preference for hierarchical phrases over serial one.

D is a derivation of the grammar, and *f(D)* and *e(D)* are the source and target strings generated by D. D is represented by a set of triples ⟨r,i,j ⟩. Here r is grammar rule which rewrites a non-terminal that spans $f(D)_i^j$ on target side. Then the weight of D is product of weight of rules used in translation, language model and word penalty,

$$w(D) = \prod_{\langle r,i,j \rangle \in D} w(r) \times p_{lm}(e)^{\lambda_{lm}} \times exp(-\lambda_{wp}|e|) \tag{2.9}$$

where, p_{lm} is language model, $\exp(-\lambda_{wp}|e|)$ is word penalty to control length on target side.

In HPBSMT, training starts with word alignment, then phrase pairs are found using Och and Ney [64], Koehn et al. [42]. Given that, all the words in phrase are consistent like in PBSMT, then set of rules of ⟨f,e,⟩is the smallest set satisfying following condition,

1. if $\langle f_i^j, e_{i'}^{j'} \rangle$is an initial phrase pair, then

   $X \rightarrow \langle f_i^j, e_{i'}^{j'} \rangle$

   is a rule.

2. if $r = X \rightarrow \langle \gamma, \alpha \rangle$is a rule and $\langle f_i^j, e_{i'}^{j'} \rangle$is an initial phrase pair such that $\gamma = \gamma_1 f_i^j \gamma_2$ and $\alpha = \alpha_{1i'}^{j'} \alpha_2$, then

   $X \rightarrow \langle \gamma_1 X_k \gamma_2, \alpha_1 X_k \alpha_2 \rangle$

   is a rule, where k is an index not used in r

above scheme generates lots of rules, which can lead to slow decoding or create ambiguity - a situation in which decoder produces many derivations that are distinct despite having the same model feature vectors. Therefore, rules generated are filtered using the following principles,

- If there are multiple initial phrase pairs containing the same set of alignment points, they keep only the smallest.
- Initial phrases are limited to a length of 10 on the source side, and rule to five (nonterminals plus terminals) on the source right-hand side.
- In the subtraction step, f_i^i must have length greater than one. The rationale is that little would be gained by creating a new rule that is no shorter than the original.
- Rules can have at most two nonterminals, which simplifies the decoder implementation. Moreover, nonterminals are prohibited that are adjacent on the target side, a major cause of spurious ambiguity.
- A rule must have at least one pair of aligned words, making translation decisions always based on some lexical evidence.

CYK parser is used as a decoder with beam search to translate a sentence. given a sentence *f*, decoder finds best derivation (or n best derivations) that generates ⟨f,e ⟩to find the target yield of the highest-probability single derivation not the whole target sentence with highest probability.

2.1.3 Factor Based Statistical Model

Koehn and Hoang [40] extended PBSMT [42] by using linguistic features/factors of source or/and target tokens. Translation of factored representation of source words to factored representation of target words is broken into following mapping steps,

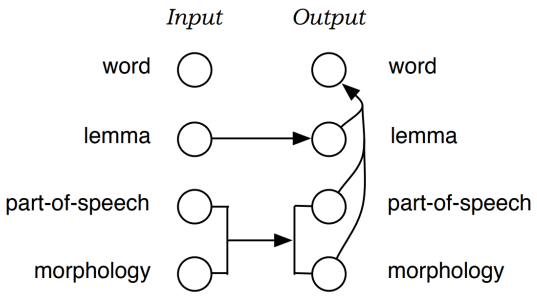


Figure 2.4: Example of Factored Model

- Translate input lemmas into output lemmas.
- Translate morphological and POS factors into output counter part.
- Generate surface forms given the lemma and linguistic factors.

2.2 Neural Models

Neural network based translation system makes use of neural networks to learn encoding and decoding functions. Encoding function takes in a sentence of language **L1** as a sequence of vectors and gives its output to a decoding function. Decoding function then generates tokens in auto-regressive manner in target language **L2** till end of sentence *<eos>* is generated.

2.2.1 Word Representation in NMT

Before going through formulation of Neural Machine Translation (NMT), let us look at how we can represent words (or parts of words) of language **L1** in a network,

- **One Hot Encoding:** As name suggest, in this type of encoding, word is represented by a vector of size V. Here V is size of vocabulary of training text in respective languages. This vector when representing a word has one element set to 1 (usually associated with index of that word in vocabulary list) and rest 0. When we have these kinds of encodings, we also have embedding weight matrix (of size Vxk, where k is some arbitrary dimension) associated with network to calculate k dimensional embeddings of words while training. This is done to reduce the number of parameters a network would have to deal with. For vocabulary

I like to go for movies.

I am going to market.

Ice creams are mostly sweet.

I must eat a lot.

VocabList	I	like	to	go	for	movies	.	am	going	market	Ice	are	mostly	sweat	must	eat	a	lot
I	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
go	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
.	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
market	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0

Figure 2.5: Example of 1 hot encoding

given in figure 2.5, if you are trying to use *market* in a network, the 1-hot encoding vector will be 18-dimension vector with all cells as zero except cell at position 10. This type of encoding entails that Neural network also needs to learn an embedding weight matrix. To avoid this overhead, many a times neural network make use of pre-trained word embeddings where words are represented using k dimension vector.

- **Context based embeddings:** These kinds of embeddings are learned using embedding learning techniques like word2vec [56] [78], glove [68] etc. on monolingual/bilingual text. These kind of embeddings (termed as pretrained when being used in NMT system) aid in improving translation systems [73] by already providing learned representations of words. To give better context to this section we give brief explanation of word2vec embedding below.

Word2vec is a multi-layered perceptron to learn k dimensional vectors for each word in the vocabulary. There are two ways of learning said vectors[2] :

- Continuous bag of words model
  This is a feed-forward neural network which predicts the word given context. Assume the following passage is part of training text to get these embeddings,
  "Indian captain Rohit Sharma was disappointed with his team's effort. In follow up QA Rohit admitted that errors on the field cost India the game in Delhi."
  Now for getting embedding of *'Rohit'* with window **4**, training input and output will be
  - Input: Indian,captain,Sharma,was, Output: Rohit

[2]Vectors and embeddings are interchangeably used

– Input: up,QA,admitted,that, Output: Rohit

In this model each input word is a one hot encoding of size V (Figure 2.6) , each of them is multiplied with a weight matrix and averaged to get a vector **h**. Now this vector **h** is then multiplied with another weight matrix to get the index of current word in one hot encoding. While training, any error network makes, backpropagates proportional error difference to change the weight matrices for whole network.

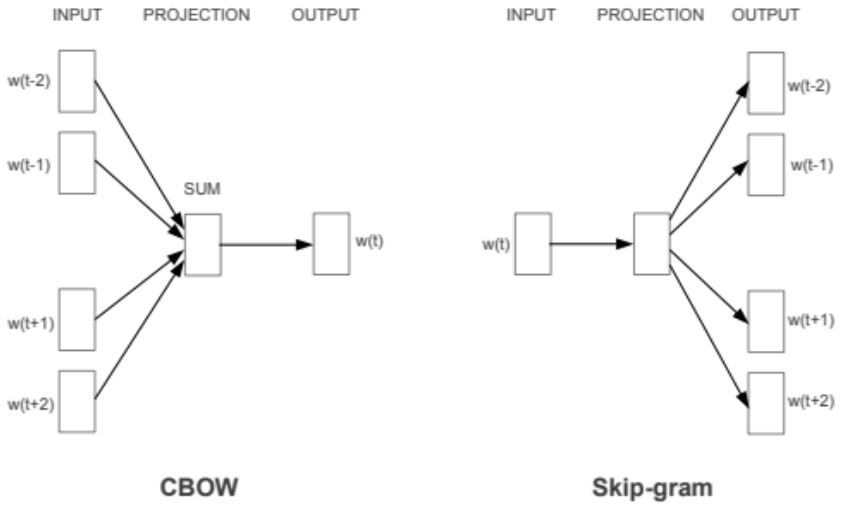


Figure 2.6: Word2vec methods

- Skip gram Model: Quite similar to above network, this network given current word this model tries to predict the context(Figure 2.6). So using above example, training pair will be
  - Input: Rohit, Output: Indian,captain,Sharma,was
  - Input: Rohit, Output: up,QA,admitted,that

2.2.2 Formulation of Neural Machine Translation

Neural Machine Translation formulation is based on two networks; sequence to sequence network [89] and attention mechanism [6]. These two networks are described below,

Sequence to Sequence Architecture

Sequence to sequence architecture [89] formulated NMT using two LSTM (Long Short Term Memory) [28] stack one after other, where one LSTM acts as encoder to encapsulate the continuous representation of sentence as a vector and other one acts as decoder by taking in this representation and generating translated sentence. As stated, earlier goal of encoder LSTM is to read input sentence, a sequence of vectors (word embeddings) x=(x_1,....x_{T_x}), and converts it into a vector **c** i.e.

$$h_t = f(x_t, h_{t-1}) \tag{2.10}$$

And

$$c = q(h_1, h_2, h_3...) \tag{2.11}$$

Where h_t is hidden state at time t, and **c** is context vector generated from these sequences. f and q are making an LSTM. Now decoder LSTM is trained to predict next word $y_{t'}$ given the context vector **c** and all the previously predicted words $y_1...y_{t'-1}$. i.e.

$$p(\mathbf{y}) = \prod_{t=1}^{T} p(y_t|\{y_1, ..., y_{t-1}\}, c) \tag{2.12}$$

Where y = (y_1,....,y_{T_y}). With LSTM it becomes,

$$p(y_t|\{y_1, ..., yt-1\}, c) = g(y_{t-1}, s_t, c) \tag{2.13}$$

Where g is nonlinear function that outputs probability of y_t and s_t is hidden state of LSTM. Figure 2.7 shows a pictorial representation of how tokens will be generated for a given input sentence.

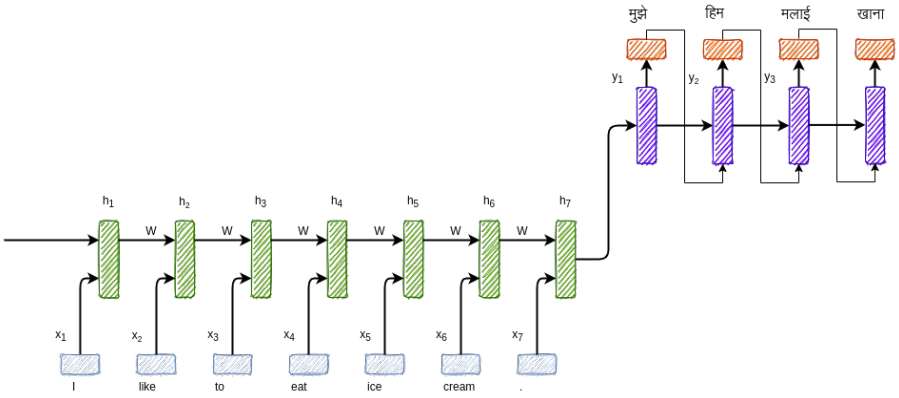


Figure 2.7: LSTM encoder processed input sentence and produce tokens on target side in auto-regressive manner using LSTM decoder.

Now the problem with this type of architecture was that it summarized the whole sentence into a fixed dimensional vector irrespective of length of the sentence. Which made it difficult for translation of longer sentences [11]. To address this issue Bahdanau et al. [6] introduced an extension to learn both translation and alignment.

NMT with Attention Mechanism

In Bahdanau et al. [6] the notion of attention was introduced in encoder decoder architecture to learn soft alignment (if one can say so), in this intuition was, when system is translating a word, it looks at neighbouring source words, previously generated translated words.

This approach doesn't try to encode the whole input sentence into a fixed vector. Instead, it encodes input sentences into sequence of context vectors and subset of these along with previously generated words are chosen to generate a word while decoding the translation. So conditional probability in equation 2.12 is rewritten as

$$p(y_t|\{y_1, ..., yt-1\}, c) = g(y_{t-1}, s_t, c_t) \tag{2.14}$$

Where s_t is hidden state for time state t in decoder, computed by

$$s_t = f(s_{t-1}, y_{t-1}, c_t) \tag{2.15}$$

Context vector c_t here is weighted sum of hidden states of sentence when it is an input in encoder LSTM.

$$c_t = \sum_{j=1}^{T_x} \alpha_{tj} h_j \tag{2.16}$$

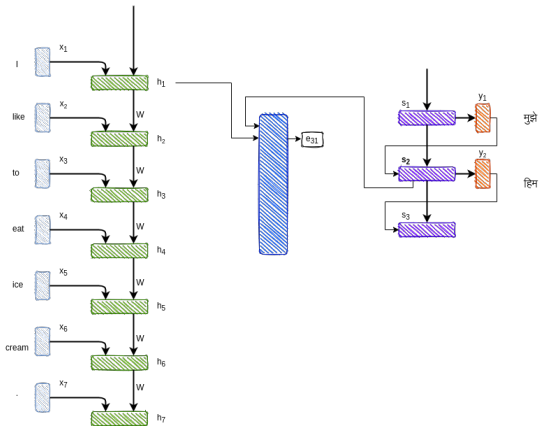
The weight α_{tj} for each h_j is calculated by

$$\alpha_{tj} = \frac{exp(e_{tj})}{\sum_{k=1}^{T_x} exp(e_{tk})} \tag{2.17}$$

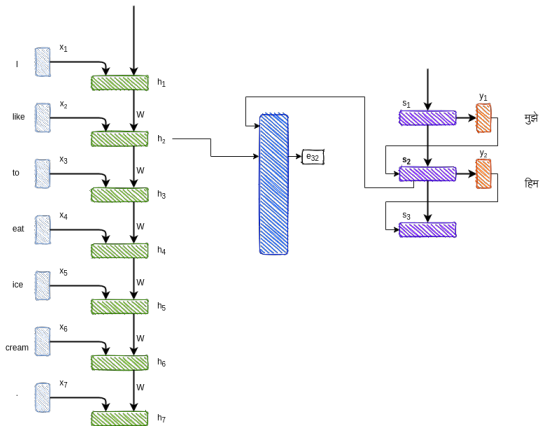
Where,

$$e_{tj} = a(s_{t-1}, h_j) \tag{2.18}$$

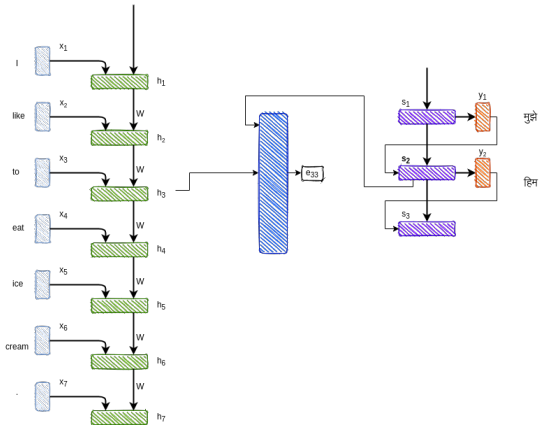
α_{tj} is the alignment model which scores how well input j is contributing while generating a word at position t, here attention is parameterized as feed-forward network. Figure 2.8 shows working of NMT with attention in step wise manner.



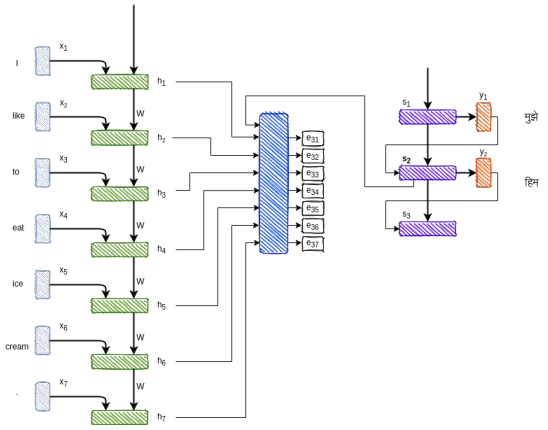
(a) Finding score using 1st source token and word previously generated. $e_{31} = a(s_2, h_1)$



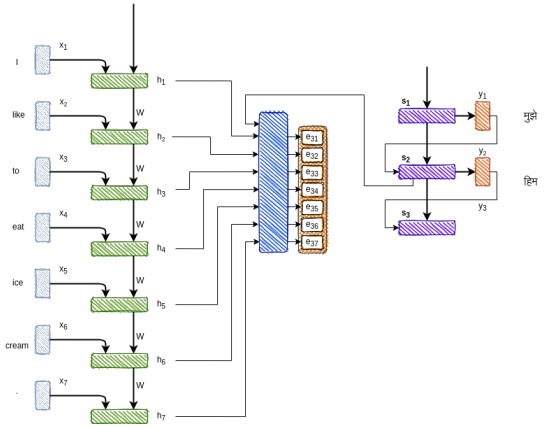
(b) Finding score using 2nd source token and word previously generated. $e_{32} = a(s_2, h_2)$



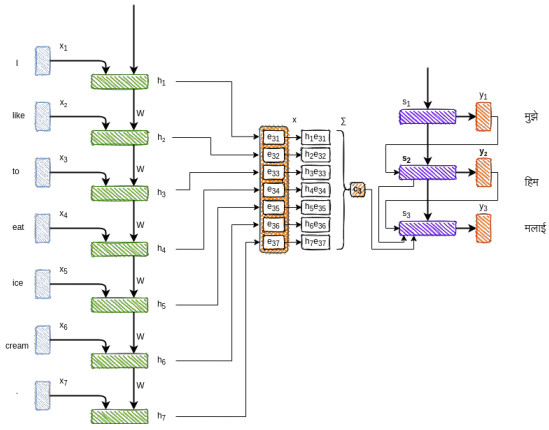
(c) Finding score using 3rd source token and word previously generated. $e_{33} = a(s_2, h_3)$



(d) Finding scores for other tokens source token and word previously generated. $e_{3i} = a(s_2, h_i)$



(e) Normalizing these scores to get attention weights for for each source token. $\alpha_{3j} = \frac{exp(e_{3j}}{\sum_{k=1}^{7} exp(e_{3k}}$



(f) Using attention weights to find the context vector for word to be generated at time step 3. $\sum_{j=1}^{7} \alpha_{3j} h_j$

Figure 2.8: Step by Step demonstration of how to find scores with various source tokens and word generated in previous time step, to find the attention weights for source tokens embeddings.

Bahdanau et al. [6] looks at every word on source side while translating. Luong et al. [54] introduced another extension in which attention was instead given to a small span of words on source side, calling it local attention. There are lot models extended from NMT to improve or tackle any issues arising in translation, some of these models are given below,

- Use of linguistic knowledge in NMT for better token generation. [82]

- Handling of OOV present in source sentence by copying the most attentive source to target. [26]
- Notion of Coverage, to make sure every source token was involved in translation. [93]
- Identifying relative importance of previous target word generated and context to generate current word. [94]
- Use of reverse translation in conjunction with forward translation for improving translation performance. [95]
- Using attention on a source word with respect to its neighbouring words (in effect finding which words in source are important to current source word), and using resultant context vector along with context vectors of other source words (calculated in similar manner) to generate translation. [97]
- Use of Document level syntax for translation. [55]

Following subsections introduce these extensions of Neural Machine Translation.

2.2.3 Factored Neural Machine Translation

Like Factored SMT (2.1.3), Factored NMT [82] make use of linguistic information of source tokens to learn translation. In this, linguistic information is embedded into embedding of source token. Suppose for a English Hindi System we are training Factored NMT (Figure 2.9). And we are using Parts-Of-Speech for source. Then for each word combined embedding we concatenate word embedding (from word embedding matrix) and pos embedding (learned similarly from POS embedding matrix). Network remains the same as NMT with attention with only distinction being that word embedding on source is comprised from multiple sources.

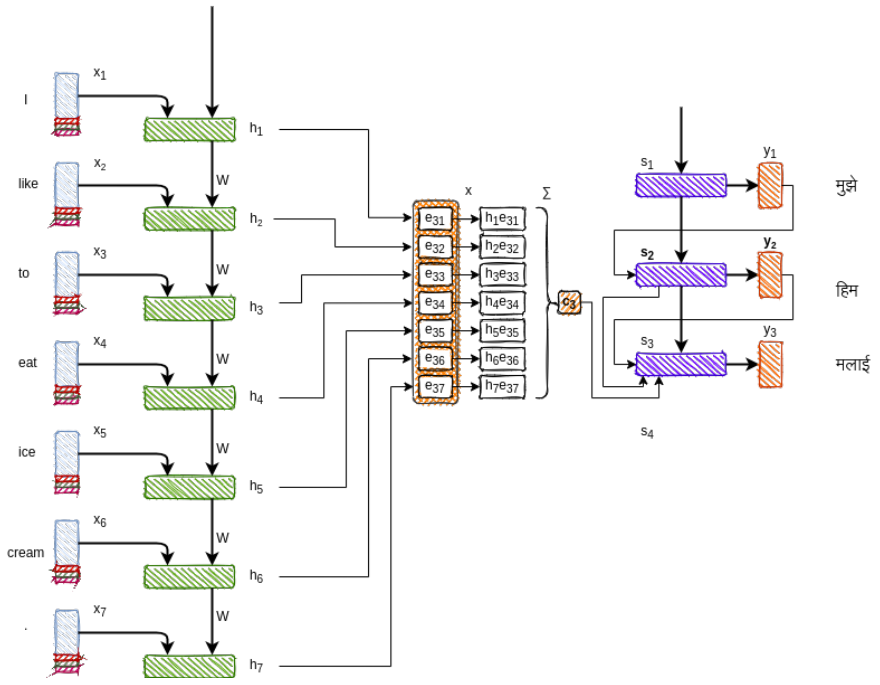


Figure 2.9: Factored NMT with usage of various features embedded together with word embedding of source token.

2.2.4 Copy Attention For NMT

There is a issue of OOV when we deal with translation. There are multiple ways of dealing with words not encountered in training. We will see one of them in later sections. An intuitive way of handling OOV in attentive networks is to make use of copy attention [26].

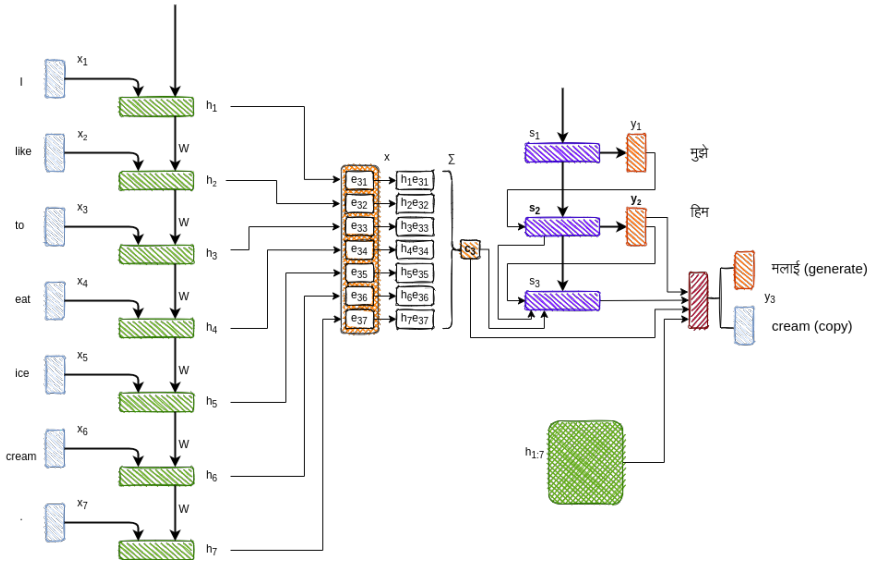


Figure 2.10: Copy Attention Model for either generating the target token or copying the source token.

In this extension, there is a small function attached to decoder, which takes in sentence embedding (hidden vectors of encoder), previously generated target word and context vector to decide whether next target word should be *generated* from target vocabulary or most attentive word in source should be *copied*. For example in Figure 2.10 generation of target word at position 3 would be as follows,

$$p(y_3|s_3, y_2, c_3, h_{1:7}) = p(y_3, \mathbf{g}|s_3, y_2, c_3, h_{1:7}) + p(y_3, \mathbf{c}|s_3, y_2, c_3, h_{1:7}) \tag{2.19}$$

Here generator and copy model are formulated as follows,

$$p(y_3, \mathbf{g}|.) = \begin{cases} \frac{1}{Z} e^{\psi_g(y_3)}, & y_3 \in V \\ 0, & y_3 \in X \cap V \\ \frac{1}{Z} e^{\psi_g(UNK)} & y_3 \notin V \cup X \end{cases} \tag{2.20}$$

$$p(y_3, \mathbf{c}|.) = \begin{cases} \frac{1}{Z} e^{\psi_c(y_3)}, & y_3 \in X \\ 0, & \text{otherwise} \end{cases} \tag{2.21}$$

where X is list of unique tokens in source, V is token list of target language, ψ_g and ψ_c are scoring functions for generate and copy modes respectively and Z is normalization constant. And ψ_g and ψ_c are formulated as follows,

$$\psi_g(y_3 = v_i) = \mathbf{v}_i^\top \mathbf{W}_o \mathbf{s}_t,\ v_i \in V \cup \text{UNK} \tag{2.22}$$

$$\psi_c(y_3 = x_j) = \sigma(\mathbf{h}_j^\top \mathbf{W}_o)\mathbf{s}_t,\ x_j \in X \tag{2.23}$$

So, functions learn what is the probability of generating a word, given that, probable word would be in target vocabulary or copying the word from most attentive source token. And make use of that to produce the next token in target.

2.2.5 Coverage Model For NMT

Coverage is implicit in SMT, there is a bookkeeping function which ensures that every entry with every source token is considered. Similar to that, in NMT to keep bookkeeping, Tu et al. [93] introduced a coverage model. In this model as words are generated, the attention weights are aggregated after each time step. And in next time step, each word's attention weights are divided by their respective aggregated attention weight. This ensures that if a word was paid more attention earlier, it will be given less attention later.

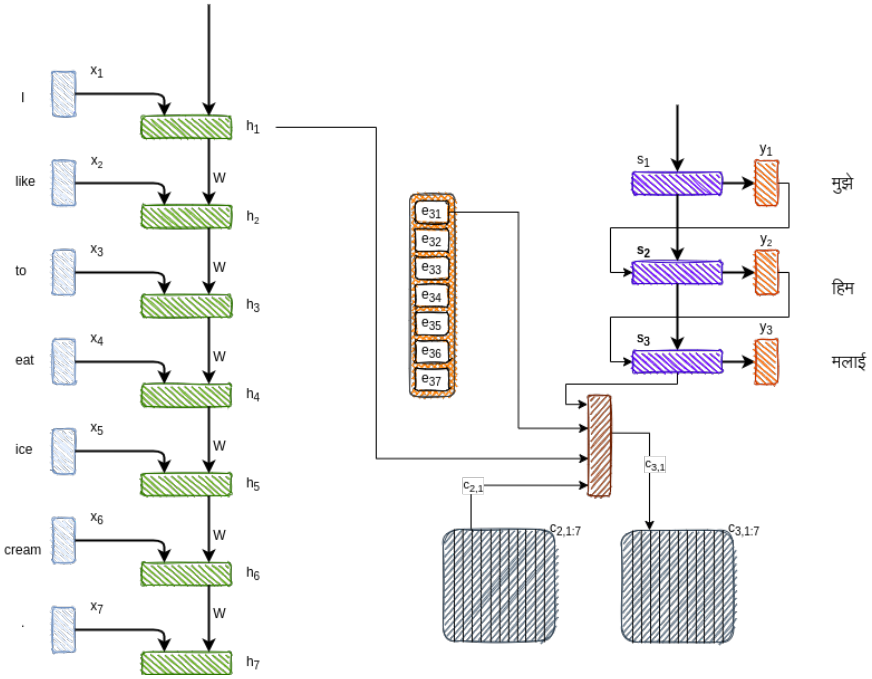


Figure 2.11: Coverage model to aggregate attention after each time step so that it can be used in translation for future steps.

For example, in Figure 2.11 if we are generating 3rd word in a sentence it would look at attention history of 1st and 2nd words generated to ensure that *I* and *ice* have less attention weights while generating मलाई.

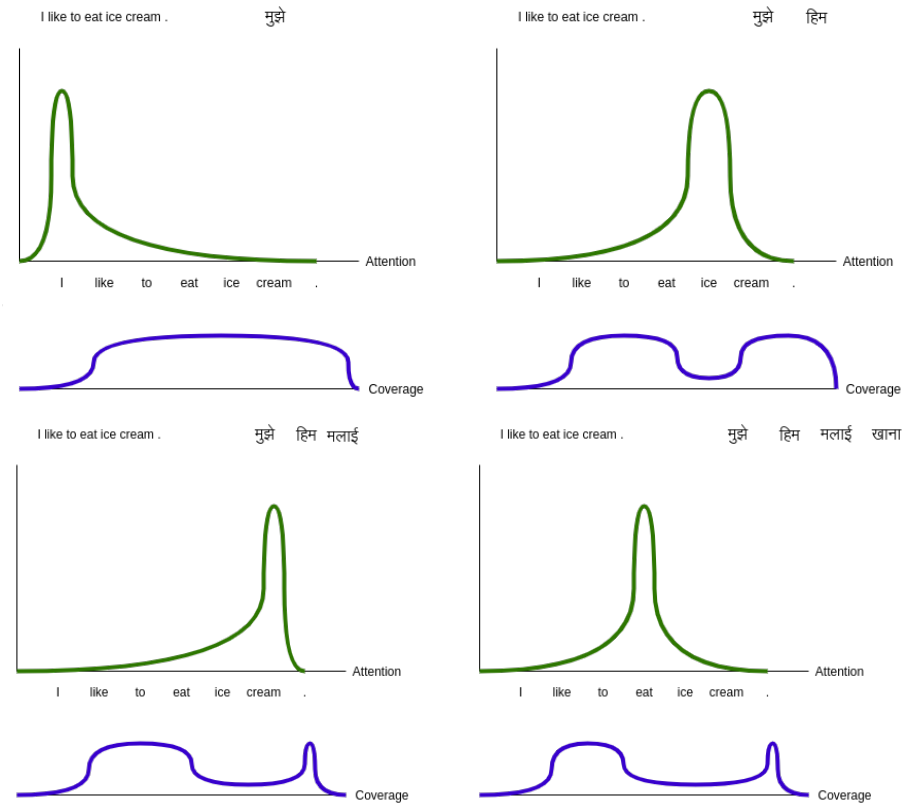


Figure 2.12: Show attention weights in green at the top and resultant coverage for each token below their respective position in graph.

Figure 2.12 shows pictorial representation of same. Here the first horizontal axis (marked Attention) points to the source words and the vertical axis marks attention weights. 2nd horizontal axis (marked Coverage) marks the resultant coverage, for this we used a simple notion of coverage being inverse of attention. So, for generation of each word with respect to attention weight would yield in *coverage mass* changing for next step, thus ensuring some form of bookkeeping to track which words have been translated.

2.2.6 Context Gate for NMT

Tu et al. [94] introduced a notion of context gates for generating output. Their rationale was, so far in attention mechanism, both context vector and previous word generated have equal weights. Instead of this, giving weight to each, context and previous word generated, would ensure that model will learn which has more importance for generating a particular word in a particular context. For Figure 2.13 context gate z_3 is modeled as,

$$z_3 = \sigma(W_z e(y_2) + U_z s_2 + C_z c_3) \tag{2.24}$$

Here $\sigma(\cdot)$ is a logistic sigmoid function, W_z, U_z, C_z are weight matrices for previously word generated, previous target hidden state and current context vector from source, respectively. There are 3 strategies to incorporate Context into NMT and they are as follows,

- Use weight to affect only Context Vector from source. In this context vector is multiplied with a weight to reduce its effect. For example, given in Figure 2.13, $s_3 = f(W_e(y_2) + U s_2 + z_3 \circ c_3)$, where $\circ$ is element-wise multiplication.
- Use weight to affect only target context from source. In this previous word generated and hidden vector of previous word is multiplied with a weight to reduce its effect. For example, given in Figure 2.13, $s_3 = f(z_3 \circ (W_e(y_2) + U s_2) + c_3)$.
- Use weight to affect both Context Vector from source and previous word. In this model takes linear interpolation between source and target context. For example, given in Figure 2.13, $s_3 = f((1 - z_3) \circ (W_e(y_2) + U s_2) + z_3 \circ c_3)$.

The intuition in this model is generation of content word should rely more on the source context and generation of a functional word should rely on the target context [94].

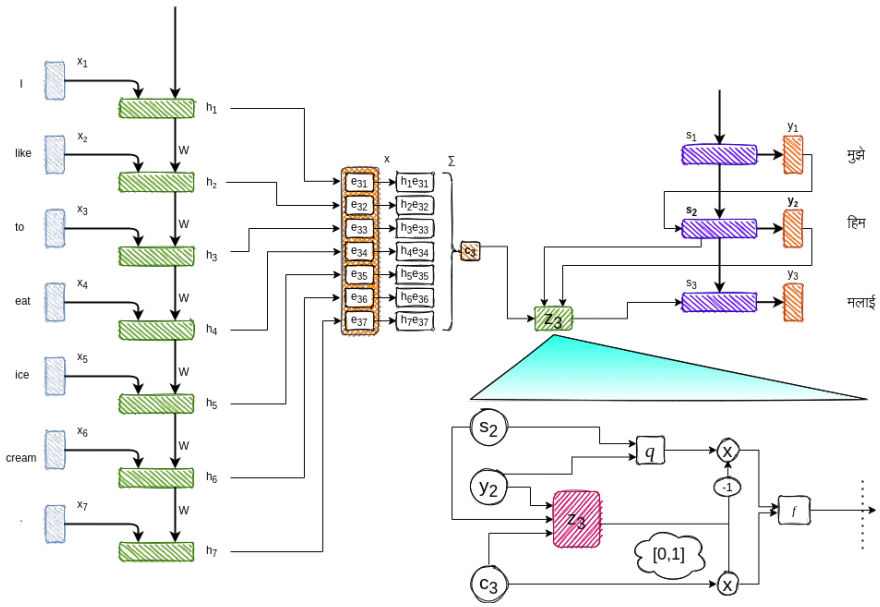


Figure 2.13: Context Gate to use in identifying relative importance of previous target word generated and context to generate current word.

2.2.7 Reconstruction of Source in NMT

Tu et al. [95] stated that NMT with attention often repeatedly translates some source words while ignoring other words. To tackle this problem, they reformulated encoder-decoder architecture to encoder-decoder-reconstructor architecture (Figure 2.14). The reconstructor, incorporated into the NMT model, would reconstruct the input source sentence from the hidden layer of the output target sentence, to ensure that the information in the source side was translated to the target side as much as possible.

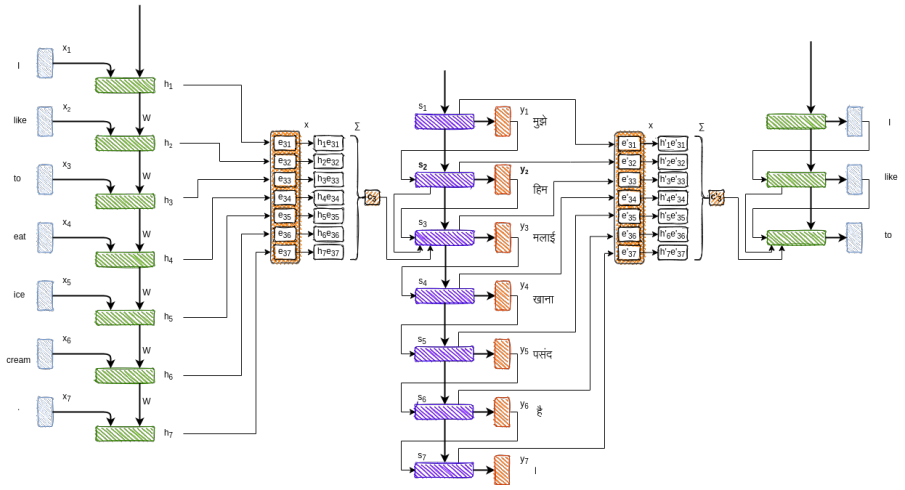


Figure 2.14: Using forward and backward translation to improve translation learning.

2.2.8 Transformer

NMT methods till now have been LSTM based encoder decoder, Vaswani et al. [97] introduced transformer, based on Lin et al. [53], which does away with LSTM architecture and uses attention mechanism entirely to learn long range dependencies between input and output. In this architecture model uses sentence to put attention on itself (self-attention), such that it can incorporate weighted embeddings of other words in sentence in current word. In Vaswani et al. [97], there are 3 different self attention network used to get 3 different embeddings for same word. These embeddings (represented by 3 different colours in Figure 2.15) are then used to get an embedding for current word. This new embedding then acts as input for a feed forward network to produce hidden vector of current word. One layer of this network consist of self-attention, feed-forward network. After going through N encoder layers, 2 self-attentive embeddings of these words are shared with decoder which goes through similar pipeline of self-attention, feed-forward network to generate words at the end.

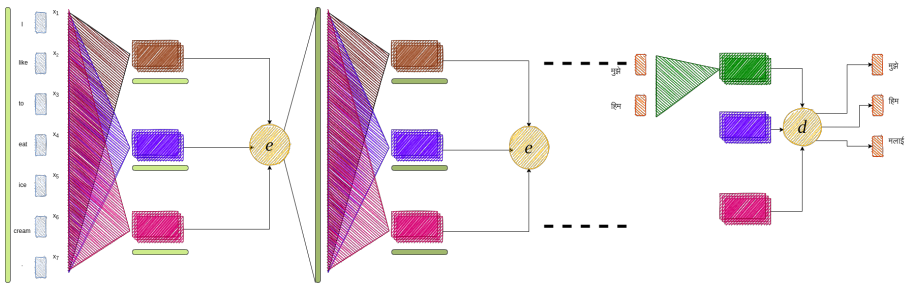


Figure 2.15: Using self-attention to encode importance of other source word for a word and decoding to generate translation.

2.2.9 Document Level Context in NMT

In MT, in general, when a system learns translation, it usually is at sentence level i.e., it makes use of sentence level context. Miculicich et al. [55] introduced document level context for translating a word by using attention mechanism in hierarchical fashion on some contextual sentence pairs. This model (Figure 2.16) for generating a word, looks at context vector of current sentence, it also looks at a document level context vector. Document level context vector is a weighted average of context vectors of x contextual sentences. Here they explored using document context and Hierarchical Attention in 3 ways

- Using Document context from source only i.e., taking weighted average of context vectors for previous x source sentences.
- Using Document context from target only i.e., using weighted average of context vectors from previous x target sentences.
- Using previous x source and target sentences with two Hierarchical Attention networks to get a document level context vector.

And they make use of context gates [94] to regulate sentence level context vector for current word and document level context.

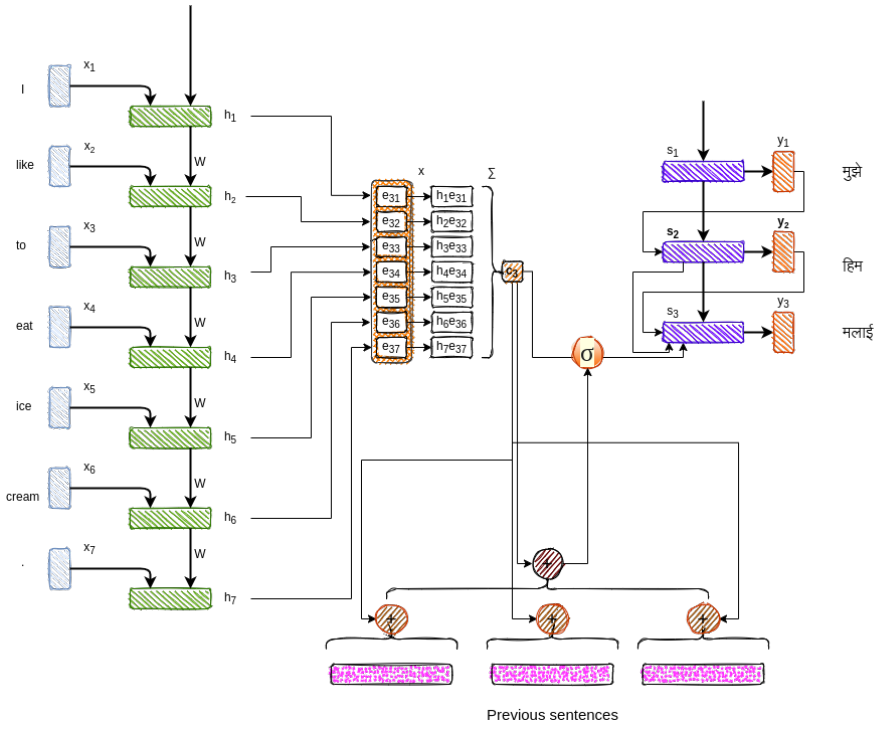


Figure 2.16: Using Document level Context by using attention mechanism in hierarchical fashion.

Hyperparameters One way of looking at neural networks is that they are highly attuned matrices and functions. Number and dimensions of matrices can affect the performance of systems. These numbers and dimensions are hyperparameters of neural networks. What we have seen as example in previous subsections was 1-layer LSTM encoder-decoder with attention mechanism network (Figure 2.8). Here additional hyperparameters which were not mentioned are size of LSTM layer, dimensions of word embeddings, learning rate[3] etc. For each task, performance of networks depends on datasets and hyperparameters. If we are using hyperparameters which was used to train high resource language [41] to train low resource bitext, our model will suffer a lot. In this case logical choice would be to do a grid search [83] and adapt neural network to dataset. This type of grid search is expensive because there can be multiple combinations increasing the number

[3]Learning rate dictates by what degree would network's weight be changed after each error

of models to be trained exponentially. E.g., if we are training with 4 different number of layers in Encoder, 4 different LSTM size and 4 different word embedding size; we have 64 models in pipeline for training. So, we must look at literature to see if there are any instances of good hyperparameter configuration for particular amount of dataset or for same language pair.

2.3 Evaluation of MT systems

We have seen various MT models, a question now arises of how do we evaluate them, which of them would suit our purpose for a particular language pair? There are multiple ways of evaluating a MT system. We can broadly classify them into,

- Automatic evaluation makes use of metrics like BLEU [66], chrF [70] etc. There is usually a reference text for translation output to be compared with. Metrics like BLEU count the n token overlap between references and translation output, whereas chrF extend this by also including m characters overlap.
- Manual Evaluation make use of humans to evaluate the translation output. There are multiple ways of manual evaluation,
  - Ranking of translations from multiple translation systems
  - Rating (giving score) to translation outputs.
  - Tagging errors in different categories like morphological, syntactical or lexical translation error.

There are various aspects to consider when we are evaluating a system. Like,

- Are we looking at evaluation of whole document, or are we looking at how translation systems handle particular aspects E.g., Named Entities in translation.
- Can evaluation be done quickly so that we can do more experiments (automatic evaluation) or we need more qualitative performance review (manual evaluation).
- Whether the metric gives adequate summary of translation system. A BLEU score gives a score for a MT system. While that has been used to claim state-of-the-art by many models, it doesn't say anything about what kind of sentences model can translate well, and what kind of sentence it can't. So, adequate summary is also another factor to look at while deciding to evaluate translation systems.

In our own work (discussed in coming sections 2.5) we have done both automatic (on whole test set) and manual evaluation (on randomly chosen 50 sentences), which gives us some insight on translation models we build.

2.4 For Indian Languages

Now let's look at models developed in literature for Indian Languages using architectures solely discussed in above sections.

Kunchukuttan et al. [47] used Moses [43][4] to build 110 systems for 11 Indian languages. They used Ramanathan et al. [75]'s and Patel et al. [67]'s rule-based reordering system which uses the Stanford parser [35] to reorder English sentence to match syntax of Indian languages. This was done because local reordering models were not adequate. And a transliteration module was used to take care of untranslated words.

Sachdeva et. al. [81] used logistic regression to select output from the translations of PBSMT and HBPSMT. Features used for logistic regression were token count (of source, target and their ratio), language model score of translation output, part of speech (POS) language model score of POS tags of translation output, number of OOV in translated sentences and entropy of parse tree using augmented version of maltparser [61]. BLEU was used as target regression value.

Sachdeva et. al. [80] used Partial least squares [1] regression to learn transformation matrices to map word embeddings (word2vec [56], glove [68]) of English and Hindi. Then they used this

[4] http://www.statmt.org/moses/

transformation matrix to transform phrase embedding of one language into another to get cosine similarity score between phrases. For a given phrase pair (**s**,**t**), they transformed each constituent word of the source phrase '**s**' to the target word space and then added the transformed word embedding to get a resultant source vector. Similar vector was computed for target phrase '**t**'. Cosine similarity between these two vectors was used as new feature for MT decoder. Another feature was transforming target phrase to source phrase in similar fashion and finding the cosine similarity score.

Dabre et. al. [14] used Moses to train factored based PBSMT. ILMT morphological analyzer and pos-tagger developed under ILMT project were used for getting linguistic features of Marathi. Freely available tool[5] was used to get same features for Hindi. It performed well for Marathi to Hindi translation system, owing to suffix splitting of Marathi whose morphemes map to appropriate words/post positions in Hindi. However, Hindi to Marathi translation proved to be more difficult due to errors in generating morphological information on Marathi side.

In all the work on Indian MT one prevalent pattern has been MT systems for Language pairs belonging to same Language family will work better (E.g., Indo Aryan to Indo Aryan) than when we try to make translation system for languages from different language families (E.g., Dravidian to Indo Aryan) [47].

2.5 Our Experiments

For our experiment to build MT systems for English-Hindi, we used SMT, NMT and transformer as training architectures. IIT Bombay English Hindi parallel corpus[6][49] was used as training dataset (after cleaning, with $\sim$ 7.5 Lakh Bitext). Testing was done on WMT 2014 English Hindi Test set[7]. For training with,

- SMT, we used Moses[8][43], we also used GIZA++ [63] to find alignments between parallel text and grow-diag-final-and method [42] to extract aligned phrases. KenLM [27] was utilized to train a LM with kneser-ney smoothing on monolingual corpus of both languages. MERT [62] was used for tuning the trained models.
- NMT, we used Opennmt[9][36] for training with default hyperparameters
- Transformer, we again used opennmt to train system using hyperparameters mentioned[10].

Table 2.2 gives BLEU scores for the same. For English to Hindi NMT and Transformer seems to be scoring better than SMT. While for Hindi to English SMT and NMT are scoring better than transformer.

Language Direction	SMT Bleu Score	NMT Bleu Score	Transformer Bleu Score
English to Hindi	9.05	12.41	13.26
Hindi to English	13.33	14.48	11.85

Table 2.2: BLEU Scores for MT systems using IIT B dataset and respective MT architectures.

For all experiments to be followed we will look at English to Hindi translation direction. Let's look at some English to Hindi samples and output of respective translation systems.

Example 1

Sentence 1 along with translation output of respective models is given in Table 2.3. In this input, if we see the sentence structure there are two clauses,

- *Female police officers* is subject.
- *will also be involved along with the PCR, Traffic Unit in that Unit* is predicate.

[5] http://sivareddy.in/downloads
[6] http://www.cfilt.iitb.ac.in/iitb_parallel/
[7] http://statmt.org/wmt14/translation-task.html
[8] https://statmt.org/moses/
[9] https://opennmt.net/
[10] https://opennmt.net/OpenNMT-py/legacy/FAQ.html

For each system we observed,

- In SMT translation output is word to word and hence seems a little weird. Also, there are instances of literal translation without taking context in effect. E.g. Female as मादा instead of महिला *that* is translated as conjunction (कि) instead of pronoun वह or इन .
- In NMT[11] translation of adverbial modifier *also* is placed at weird position to make translation off.
- Transformer has better translation than other two systems.

Input	Female police officers will also be involved along with the PCR , Traffic Unit in that Unit .
Reference	इन जवानों में पीसीआर , ट्रैफिक यूनिट सहित महिला पुलिसकर्मी शामिल होंगी ।
SMT	मादा पुलिस अधिकारियों को भी शामिल किया साथ ही पीसीआर , यातायात इकाई में कि यूनिट है ।
NMT	महिला पुलिस अधिकारी इस यूनिट में पीसीआर , ट्रैफीक यूनिट के साथ भी शामिल होंगे .
Transformer	महिला पुलिस के अधिकारियों को भी इस इकाई में पीसीआर , यातायात इकाई के साथ शामिल किया जाएगा .

Table 2.3: Translations for Sample 1

Example 2

Sentence 2 along with translation output of respective models is given in Table 2.4. In this input, if we see the sentence structure there are two clauses,

- *I look forward to reviewing the report of the investigation with UK ministers next year* is subject.
- *and together decide on the next steps* is predicate.

For each system we observed,

- In SMT translation output is word to word and hence word order seems off. E.g. रिपोर्ट की जांच instead of जांच की रिपोर्ट or निर्णय पर अगले कदम instead of अगले कदम निर्णय पर .
- In NMT translation of 1st clause was good, but there was an omission error, there was no translation of *together*, adverbial modifier *also* is placed at weird position to make translation off.
- Transformer went through text degradation from start, later clause was partially translated well with omission error when concerning translation of *together*.

Input	I look forward to reviewing the report of the investigation with UK ministers next year and together decide on the next steps .
Reference	मैं अगले वर्ष ब्रिटिश सरकार के मंत्रियों के साथ जांच रिपोर्ट की समीक्षा करने और आगे की कार्रवाई पर साथ मिलकर निर्णय लेने के लिए तत्पर हूँ ।
SMT	मुझे आशा है की समीक्षा कर रिपोर्ट की जांच के साथ में यू . के मंत्रियों की अगले वर्ष और साथ में यह निर्णय पर अगले कदम उठाए हैं ।
NMT	मैं अगले साल ब्रिटेन के मंत्रियों के साथ जांच की रिपोर्ट की समीक्षा करने की उम्मीद करता हूं और अगले चरण पर निर्णय करता हूं .
Transformer	मैं अगले वर्ष यू . के . के . के . के . के . के . के साथ जांच की रिपोर्ट की समीक्षा करता हूं और अगले कदम के बारे में निर्णय लेता हूं .

Table 2.4: Translations for Sample 2

[11] NMT is used interchangably with NMT with attention

Example 3

Sentence 3 along with translation output of respective models is given in Table 2.5. In this input, if we see the sentence structure there are two clauses,

- *When the Municipal boards in the district come under Trunamool party control* is subject.
- *due to the change of party, then the new members of the party are made very welcome in Kolkata at the Trunamool Bhavan .* is predicate.

For each system we observed,

- In SMT translation same issues like in previous two outputs was observed.
- In NMT translation was better than rest of the systems but there was a post position error (should have been सदस्यों instead of सदस्य) and an omission error (Missing का at underscore position).
- Transformer conveyed information but there were multiple omission errors, E.g. translation of Kolkata, Trunamool Bhavan etc.

Input	When the Municipal boards in the district come under Trunamool party control , due to the change of party , then the new members of the party are made very welcome in Kolkata at the Trunamool Bhavan .
Reference	जिलों में नगरपालिका बोर्ड पर जब दल बदल के कारण तृणमूल का कब्जा होता है तो पार्टी में शामिल होने वालों को कोलकाता स्थित तृणमूल भवन में भव्य स्वागत होता है ।
SMT	जब नगर परिषद को जिले के अंतर्गत आ Trunamool पार्टी के नियंत्रण , में बदलाव के पक्ष में , फिर नए सदस्यों की पार्टी ने उनका स्वागत बहुत ही में कोलकाता को Trunamool भवन है ।
NMT	जब जिले में नगर बोर्डों का नियंत्रण Trunamool पार्टी के अधीन होता है तो पार्टी के नये सदस्य _ कोलकाता के Trunamool भवन में बहुत स्वागत करते हैं .
Transformer	जब जिले के नगर मंडल Trunamool पार्टी नियंत्रण के अन्तर्गत आते हैं तो पार्टी के नये सदस्यों का स्वागत भवन में हुआ .

Table 2.5: Translations for Sample 3

Then we further analysed 50 outputs manually. Two kinds of analysis were done,

- Quality Rating: each output was rated on its adequacy (translation information from source) and fluency (how fluent is Hindi in the output). For each sentence we gave a score in adequacy and fluency.

  For adequacy we looked at this question and scored according to scores given below. Does the output convey the same meaning as the input sentence? Is part of the message lost, added, or distorted?

  - 5 all meaning
  - 4 most meaning
  - 3 much meaning
  - 2 little meaning
  - 1 none

  For fluency, is the output good fluent Hindi? This involves both grammatical correctness and idiomatic word choices.

  - 5 flawless Hindi
  - 4 good Hindi
  - 3 non-native Hindi
  - 2 disfluent Hindi
  - 1 incomprehensible

- Named Entity (NE) coverage: For NEs in input sentences, how many NE were translated correctly (or source NE was placed in correct position, if translation was not found) and how many were not translated correctly.

We also looked at the result from perspective of sentence length. Sentences were divided into two categories - with token length of input sentence less than 20 and second group was of longer sentences. This was done to see how MT systems perform for shorter and larger sentences. Figure 2.17 shows the average score of fluency and adequacy for all sentences, for sentences with token length less than 20 and for sentences with token length 20 or greater. Table 2.6 shows NE coverage in similar fashion.



Figure 2.17: Average Fluency and Adequacy score for each translation systems with respect to input sentence length.

Sentences (# of NE)	SMT		NMT		Transformer	
	NE Trans	NE Not Trans	NE Trans	NE Not Trans	NE Trans	NE Not Trans
all sent (82 NE)	**74**	**8**	57	25	45	37
tok len <20 (21 NE)	**20**	**1**	18	3	13	8
tok len >= 20 (61 NE)	**54**	**7**	40	22	32	29

Table 2.6: NE Coverage for each system

We can summarise our observations as follows,

- SMT has better NE coverage than NMT and transformer in other words OOV issues for Neural models.
- Neural models have better fluency.
- If we compare all the metrics (adequacy, fluency, NE coverage) we see that, the translations of longer input sentences suffer when compared with translations of short sentences.

There are pre-processing steps which can be done to alleviate these translations. Let us look at what can be done as a pre-processing step to a sentence before sending it for translation in the following section.

Chapter 3

Preprocessing

Preprocessing in Machine Translation is done to input text to make it MT ready. By MT ready we mean, it can be used for training or testing. Steps like tokenization, normalization and sometimes lower-casing of Roman scripts are implicit preprocessing steps done to data set for MT. There are techniques which go a bit further and try to preprocess input such that translation system can learn/do better translation, E.g splitting of suffix of a word in Marathi to make Marathi to Hindi translation system learn better [14]. For this report we will look at the following preprocessing steps,

- Splitting of words into subwords (E.g. in Table 3.1). As we will see in subsection 3.1, this can help in case of OOVs, Data scarcity etc. And how these have been used for Indian languages.
- Reordering of English to match Indian language syntax. (E.g. in Table 3.1)
- Splitting long sentences into short segments and translating them.

English Text	Speaking on the occasion , Bajirao Sathe said that he was undergoing treatment at the CPR Hospital .
BPE	Speaking on the occasion , Bajirao Sa@@ the said that he was undergoing treatment at the C@@ P@@ R Hospital .
Morfessor	S +peak +ing on the occasion , Bajirao Sa +the s +aid th +at he was under +go +ing treat +ment at the C +PR Hospital .
CFILT reorder	the occasion on Speaking , Bajirao Sathe that he treatment the CPR Hospital at undergoing was said .

Table 3.1: Different Preprocess on Text

And then subsection 3.4 shows result of experiments we have done with some of these techniques.

3.1 Subwords

There have been multiple ways of splitting a word into smaller components. We will look at some of prominently used methods.

Byte Pair Encoding

Byte Pair Encoding (BPE) is a dictionary encoder that incrementally finds a set of symbols such that the total number of symbols for encoding the text is minimized. Sennrich et al. [84] made use of BPE to alleviate translation performance, especially when dealing with OOVs. In this method, vocabulary is initialized as a list of sequence of characters with a special end of word symbol. Find the most frequent bigram character and replace them with single token as we can see in Figure 3.1. Now vocab list is updated accordingly (3.1), this is *1 merge* operation. Now iteratively do this again n times. In BPE model, n is the hyperparameter and any new input is segmented according to vocab list found after n merge operations.

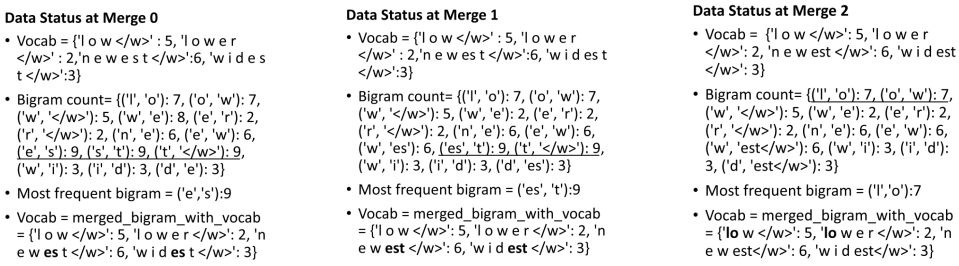


Figure 3.1: Iterative algorithm to find the subword using BPE, most frequent bigram at each merge is underlined.

Number of merge operation as stated earlier is a hyperparameter, so optimal number of merge operations are dependent on system training setting[1]. Ding et al. [22] worked on identifying factors which dictate number of merge operations in training setting. And they found that transformers work well with 0.5K to 2K merge operations. Number of merge operations for LSTM based NMT with attention is more case to case specific. Joining vocabulary of two languages to train BPE and training BPE for each language separately produces equivalent result in translation. And they found that type token ratio and alignment ratio[2] do not have any effect on choice of merge operation. Training involving languages of Fusional, Introflexive and Agglutinative type do get affected by varying hyperparameter of BPE.

Unigram LM

Unigram LM [45] encodes most probable sequence of characters together so that model can restrict number of vocab to certain value.

Morfessor

Morfessor [86] is another method for learning segmentation, it uses a local search and utilizes recursive splitting. They choose the segmentation, which is optimal given across various tokens, with threshold on how many suffixes they can identify.

Orthographic Syllable

Another route for segmentation is usage of orthographic syllable [46]. Orthographic syllable is a sequence of one or more consonants followed by vowel. This has been used with good results for related Indian languages and to some extent for languages across families in Indian language diaspora.

For Indian Languages

Pimpale et al. [69] made Marathi to Hindi SMT system which split the compound words to tackle the morphological richness of Marathi. A Splitting algorithm was used which checked if a given Marathi word contains a suffix from list of commonly used suffixes. They found translation systems trained on suffix-splitted data outperform baseline.

Dabre et al. [14] used Moses to train factored based PBSMT. ILMT morphological analyzer and postagger developed under ILMT project were used for getting linguistic features of Marathi. Freely available tool[3] was used to get same features for Hindi. It performed well for Marathi to Hindi translation system, owing to suffix splitting of Marathi whose morphemes map to appropriate words/post positions in Hindi. However, Hindi to Marathi translation proved to be more difficult due to errors in generating morphological information on Marathi side.

Kunchukuttan and Bhattacharyya [46] explored the use of the orthographic syllable (OS), which is a variable-length consonant-vowel sequence, as a basic unit of translation between related languages [7] which use abugida or alphabetic scripts. They found that using OS as basic unit

[1]By training setting we mean - number of bitext, training architecture, language pair involved etc.
[2]# of one-to-many alignments/# of many to one alignments
[3]http://sivareddy.in/downloads

aids in translation when significantly morphologically rich languages are involved. Also, it helps in translation between different language families (Indo Aryan and Dravidian in this case), while addressing data sparsity issue of low resource morphologically rich languages in translation to some extent.

3.2 Reordering

Although there is a reordering model in SMT, the difference in syntax between English and Indian languages still introduces error. In some work pre-ordering English to match target language syntax as a preprocessing step has built better translation systems by reducing structural complexity in syntax difference. Some of the ways this kind of pre-ordering[4] has been done is given below.

Using Syntax Information of source and/or target sentences

Collins et al. [12] described a method for incorporating syntactic information in SMT systems. The first step of the method was to parse the input and the second step was to apply a series of transformations to the parse tree, effectively reordering the surface string on the source language side of the translation system to match word order that is closer to the target language word-order. Patel et al. [67] and Ramanathan et al. [75] are a rule-based systems which takes in parsed English sentences and reorder them to match Indian language syntax. This has shown promising results in various language pairs one of the most widely used work which showed this is Shata-anuvadak[47] introduced earlier, they built PBSMT based English to Indian (10) languages translation systems and English was reordered to match Indian language syntax with good results over Canonical English to Indian Languages.

Using Parallel text

Bracketing Transduction Grammar (BTG) [98] has been used to learn reordering model on parallel text to pre-order Source language to match target language syntax [20], [59], [58]. BTG is a binary synchronous context-free grammar with only one non-terminal symbol and has three types of rules (Figure 3.2): *Straight* which keeps the order of child nodes, *Inverted* which reverses the order, and *Terminal* which generates a terminal symbol. So for input we have seen earlier English should be preordered as given in Figure 3.3. BTG for the same is given in Figure 3.4.

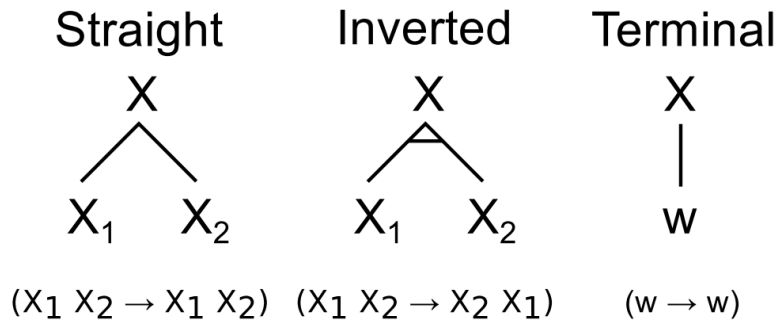


Figure 3.2: Bracketing transduction grammar

There have been different ways of modeling this from alignments of parallel text. DeNero and Uszkoreit [20] suggested that unsupervised grammar induction can be used to create source-sentence parse structure for use in translation as a part of a pre-ordering-based translation system. DeNero and Uszkoreit [20] build two models - parsing and tree ordering. Neubig et al. [59] did reordering by treating the discriminative parser's derivation tree as a latent variable in a model that is trained to maximize reordering accuracy. Nakagawa [58] used incremental topdown parsing method with Perceptron for pre-ordering based on Bracketing Transduction Grammar (BTG).

[4] we will be using pre-ordering and reordering interchangeably

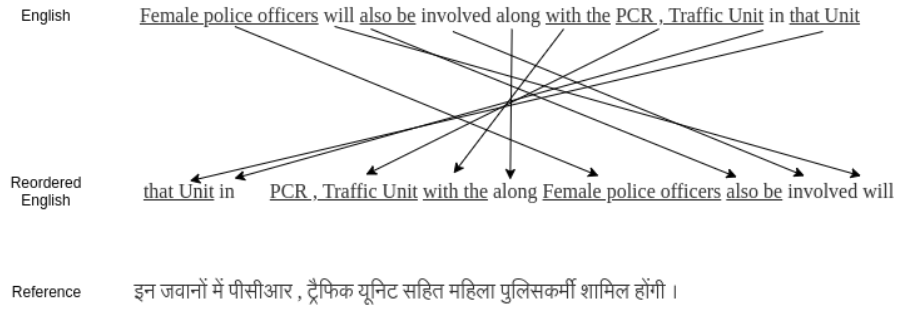


Figure 3.3: Pre-ordering For English Hindi translation. To keep diagram uncluttered, some words following same order after pre-ordering have been grouped together and marked as underlined.

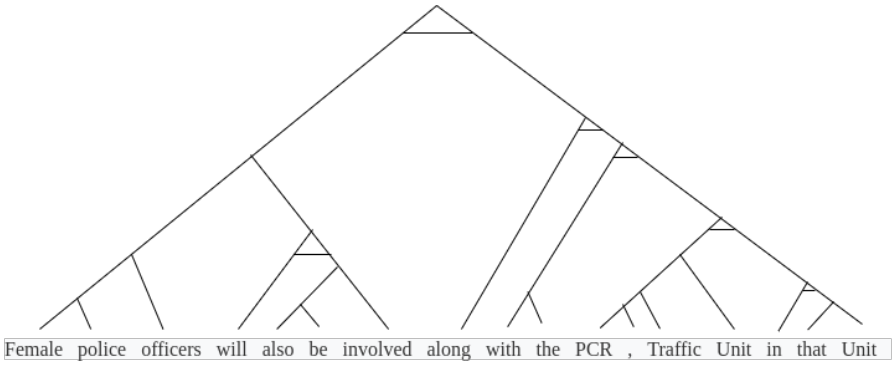


Figure 3.4: BTG tree representing the pre-ordering

3.3 Segment and Translate

There has been multiple work which observed disfluency, word drop, repetition in translation output of longer sentence by SMT, NMT systems and used some ways to deal with these, as seen in previous section. But there is another interesting perspective to look from, of segmenting the input sentences, translating them separately and then joining the sub-outputs to get translation. In this subsection we would be looking at some of them, how they are modeled with MT system, what are the resources they need and what can be issues in them if any. Primarily while going through literature, we came across two kinds of methodology for segmentation,

- Explicit segmentation: Here there is a segmentation model which segments sentences into short sub-sentences for translation. After translation, these translated sub-sentences are stitched together to give an output. [88], [72], [44]
- Implicit segmentation: Here model requires features like syntax, clause information to implicitly segment translation and stitch sub-sentence translation together. There is no explicit stitching, translation is done in 2-step manner. [32]

Explicit Segmentation

Pouget-Abadie et al. [72] introduced as segmentation model for sequence-to-sequence network on English French language pair. This model tried finding a phrase in source such that translation score from that phase to target word and vice versa was high. So, the target of the whole training was to maximize finding phrases in this manner. But this was computationally expensive as the model has to go through each possible sequence of sub-sequence in input sentence.

Sudoh et al. [88] worked on English to Japanese language pair. Sentence was segmented into clauses using parser from source language and non-terminals were used as a placeholder for embedded clauses. These clauses were aligned with the words on the target side using alignment from PBSMT model. They've made use of standard SMT method for training. So, the model translates the main clause, then translates non terminals in a cascading manner. So here the model is not trying to reorder the whole sentence, instead it works at clause level. This worked better than

baseline even when the number of clauses increased. But there was a downside of disfluencies in final translation because in parts you are translating them piece by piece and there is no Oracle to amend the final translation.

Kuang and Xiong [44] worked on Chinese to English. They made use of alignments from SMT to train a classification model; that is, can a sentence split at position k, such that, translation of subsequences on either side of k would be optimal. They try to find the longest subsequence possible. And then, there is a reordering model which models whether the translation of two neighbouring subsequences can be concatenated in straight order; if it can then keep them as they are otherwise find better order of subseqeunces. This seems a bit more promising than other methods albeit there is possibility of misalignments but if we can make use of alignments properly then this can be useful for longer sentences.

Implicit Segmentation

Ishiwatari et al. [32] developed 2-step hierarchical decoder for English to Japanese. This model entails that there is a resource/tool to identify chunks on target sentences. While translating, context vector generates a hidden vector known as chunk vector. Chunk vector is trained to generate words till end-of-chunk tag is reached. And then like in Bahdanau et al. [6] chunk vector is forwarded to next time step to generate next chunk.

For our experiments we have used subwords and reordering as preprocessing steps. In following subsection, we will look at some of English to Hindi translation systems, their performances and some outputs.

3.4 Our Experiments

We performed two types preprocessing for English to Hindi translation system. These were as follows,

- Subword: We used BPE [84] with various merge operation values ranging from 10K to 100K with step size of 10K for both English and Hindi. After automatically evaluating these 10 systems we found that NMT with attention trained on bitext segmented using 10K merge operation, on both English and Hindi, gave best output.
- Reordering: First, we used CFILT preordering tool[5] [67] to reorder English to match Hindi Syntax and train two systems: NMT with Attention and Transformer. Second, we made use of alignments from mgiza[6](another tool apart from GIZA++ which can be used to find alignments) and reordered English accordingly. Then we again found the alignment between reordered English and Hindi and continued this till there was no change in English side. Once we got reordered English, we treated reordering as sequence-to-sequence learning task with English as source and reordered English as target language and trained system. But there's an inherent weakness here, automatic word alignment is not always reliable so does the system output.

After keeping all other hyperparameters same, we had the following systems,

- NMT 10K BPE On Both Sides: NMT with Attention system, trained on bitext which was segmented using 10K merge operations.
- NMT trained on CFILT Reordered English Hindi: Train NMT on preordered English - Hindi.
- Transformer trained on CFILT Reordered English Hindi: Train transformer on preordered English - Hindi.
- NMT trained on NMT reordered English Hindi: First reorder English using NMT reordering system described above. And then train Reordered English-Hindi.

Table 3.2 gives BLEU scores for all the preprocessing done. We see the maximum gain in NMT with 10K BPE. We also found Neural models performance was affected when reordered English was used, regardless of whether reordering system was rule based or neural trained. Let's look at the translations of examples given in 2.5 with these systems,

[5] https://www.cfilt.iitb.ac.in/static/download.html
[6] https://github.com/moses-smt/mgiza

Model	BleuScore
SMT	9.05
NMT	12.41
Transformer	13.26
NMT 10K BPE On Both Sides	**13.73**
Cfilt Reordered En Hi NMT	11.01
Cfilt Reordered En Hi Transformer	10.74
NMT Reordered En Hi NMT	10.93

Table 3.2: BLEU Scores for English To Hindi Translation systems

Example 1

For example 1, outputs are given in Table 3.3. And based on them our observations are,

- NMT with 10K BPE produces good output without any degradation.
- Reordering was a bit off which can be seen in translation outputs for both NMT with attention and transformer systems.
- NMT based reordering got a decent translation.

Input	Female police officers will also be involved along with the PCR , Traffic Unit in that Unit .
Reference	इन जवानों में पीसीआर , ट्रैफिक यूनिट सहित महिला पुलिसकर्मी शामिल होंगी ।
SMT	मादा पुलिस अधिकारियों को भी शामिल किया साथ ही पीसीआर , यातायात इकाई में कि यूनिट है ।
NMT	महिला पुलिस अधिकारी इस यूनिट में पीसीआर , ट्रैफीक यूनिट के साथ भी शामिल होंगे .
Transformer	महिला पुलिस के अधिकारियों को भी इस इकाई में पीसीआर , यातायात इकाई के साथ शामिल किया जाएगा .
NMT 10K BPE On Both Sides	इन जवानों में पीसीआर , ट्रैफिक यूनिट सहित महिला पुलिसकर्मी शामिल होंगी ।
CFILT Reordered English	Female police officers also the PCR , that Unit in Traffic Unit with involved along be will .
NMT trained on CFILT Reordered English Hindi	इस यूनिट के यातायात यूनिट के साथ – साथ महिला पुलिस के अधिकारियों का भी PCR होगा . { involvement's translation is missing}
Transformer trained on CFILT Reordered English Hindi	अधिनियम की धारा में महिला पुलिस अधिकारियों को भी शामिल किया जाएगा . { parts of predicate are missing}
NMT Reordered English	that Unit in PCR , Traffic Unit with the along Female police officers also be involved will .
NMT trained on NMT Reordered English Hindi	उस इकाई में PCR , यातायात इकाई के साथ , महिला पुलिस अधिकारी भी शामिल होंगे . { decent translation}

Table 3.3: Translation Output of Example 1 after preprocessing

Example 2

For example 2, outputs are given in Table 3.4. And based on them our observations are,

- NMT with 10K BPE produces good output without any degradation.
- NMT trained on CFILT Reordered English Hindi produced decent output.
- Transformer trained on CFILT Reordered English Hindi produced fluent output but there was coverage issue like translation of *look forward*. This can be due to reordering tool, putting them far apart while producing reordered English.
- As for NMT based reordering it produced investigation ministers while reordering English so there therein lies the scope of error; in learning reorder itself; here it learnt it incorrectly or tried to create the word incorrectly and hence that error was propagated forward while translation.

Input	I look forward to reviewing the report of the investigation with UK ministers next year and together decide on the next steps .
Reference	मैं अगले वर्ष ब्रिटिश सरकार के मंत्रियों के साथ जांच रिपोर्ट की समीक्षा करने और आगे की कार्रवाई पर साथ मिलकर निर्णय लेने के लिए तत्पर हूँ ।
SMT	मुझे आशा है की समीक्षा कर रिपोर्ट की जांच के साथ में यू . के मंत्रियों की अगले वर्ष और साथ में यह निर्णय पर अगले कदम उठाए हैं ।
NMT	मैं अगले साल ब्रिटेन के मंत्रियों के साथ जांच की रिपोर्ट की समीक्षा करने की उम्मीद करता हूं और अगले चरण पर निर्णय करता हूं .
Transformer	मैं अगले वर्ष यू . के . के . के . के . के . के साथ जांच की रिपोर्ट की समीक्षा करता हूं और अगले कदम के बारे में निर्णय लेता हूं .
NMT 10K BPE On Both Sides	मैं अगले वर्ष ब्रिटिश सरकार के मंत्रियों के साथ जांच रिपोर्ट की समीक्षा करने और आगे की कार्रवाई पर साथ मिलकर निर्णय लेने के लिए तत्पर हूँ ।
CFILT Reordered English	I forward UK ministers with the investigation of the report next year reviewing to look and together the next steps on decide .
NMT trained on CFILT Reordered English Hindi	मैं अगले वर्ष ब्रिटेन के मंत्रियों के साथ जांच की रिपोर्ट की समीक्षा करने और अगले कदम पर निर्णय लेने के लिए उत्सुक हूं . {decent translation}
Transformer trained on CFILT Reordered English Hindi	मैं अगले साल ब्रिटेन के मंत्रियों के साथ अन्वेषण की रिपोर्ट की समीक्षा करता हूं और अगले कदम तय करता हूं . {look forward – missing translation}
NMT Reordered English	I next year investigation ministers with investigation of the report to reviewing the look forward and next steps on the together decide .
NMT trained on NMT Reordered English Hindi	मैं आगामी वर्ष जांच मंत्रियों के साथ जांच की समीक्षा करने और अगले चरण पर एक साथ विचार करने के लिए कदम उठाए . {UK missing, जांच मंत्रियों – wrong translation. Look forward is missing}

Table 3.4: Translation Output of Example 2 after preprocessing

Example 3

For example 3, outputs are given in Table 3.5. And based on them our observations are,

- NMT with 10K BPE produces decent translation but, if we can post edit postposition or verb Inflexion, translation would have been better.
- Translation systems trained on CFILT based reordering produced English produced suboptimal translations in all 3 systems.

- Also, as was the case earlier NMT reordering system has coverage issue; not every word in input is getting reproduced.

Input	When the Municipal boards in the district come under Trunamool party control , due to the change of party , then the new members of the party are made very welcome in Kolkata at the Trunamool Bhavan .
Reference	जिलों में नगरपालिका बोर्ड पर जब दल बदल के कारण तृणमूल का कब्जा होता है तो पार्टी में शामिल होने वालों को कोलकाता स्थित तृणमूल भवन में भव्य स्वागत होता है ।
SMT	जब नगर परिषद को जिले के अंतर्गत आ Trunamool पार्टी के नियंत्रण , में बदलाव के पक्ष में , फिर नए सदस्यों की पार्टी ने उनका स्वागत बहुत ही में कोलकाता को Trunamool भवन है ।
NMT	जब जिले में नगर बोर्डों का नियंत्रण Trunamool पार्टी के अधीन होता है तो पार्टी के नये सदस्य _ कोलकाता के Trunamool भवन में बहुत स्वागत करते हैं .
Transformer	जब जिले के नगर मंडल Trunamool पार्टी नियंत्रण के अन्तर्गत आते हैं तो पार्टी के नये सदस्यों का स्वागत भवन में हुआ .
NMT 10K BPE On Both Sides	जिलों में नगरपालिका बोर्ड पर जब दल बदल के कारण तृणमूल का कब्जा होता है तो पार्टी में शामिल होने वालों को कोलकाता स्थित तृणमूल भवन में भव्य स्वागत होता है । {Wrong Post position}
CFILT Reordered English	When the district in the Municipal boards Trunamool party control under , party of the change to due come , the party of then the new members the Trunamool Bhavan at very welcome Kolkata in made are .
NMT trained on CFILT Reordered English Hindi	जब जिला बोर्डों को Trunamool पार्टी नियंत्रण के अधीन पार्टी के नियंत्रण में लाया जाता है तो पार्टी के नये सदस्य _ कोलकाता में Kolkata भवन में बहुत स्वागत करते हैं . {Missing का at underscore position – post position Error, and सदस्य should be plural, since system cant translate trunamool , kolkata most weighted word for it}
Transformer trained on CFILT Reordered English Hindi	जब जिले में नगर बोर्डों का नियंत्रण पार्टी में होता है तो पार्टी के नए सदस्य कोलकाता में very भवन में स्वागत करते हैं . {Some information are missing – Kolkata, Trunamool Bhavan}
NMT Reordered English	When the district in the Municipal boards Trunamool party control , due under party of change to the , then party of the new members the Trunamool Bhavan at the Kolkata in welcome very welcome are made .
NMT trained on NMT Reordered English Hindi	जब जिले में नगर बोर्डों के दल नियंत्रण के कारण पार्टी के परिवर्तन के कारण पार्टी के नए सदस्यों को welcome भवन में आयोजित किया जाता है तो उन्हें कोलकाता में स्वागत में बहुत स्वागत है . {Repetition}

Table 3.5: Translation Output of Example 3 after preprocessing

As was the case in the last section, we analysed 50 outputs for 4 systems manually. We looked at both quality rating (adequacy and fluency) and NE coverage to understand how systems are working with different preprocessing steps. As was the case earlier we looked at this analysis with 3 different sentence length groups (All, sentence with token length less 20, sentence with token length greater or equal to 20). Figure 3.5 shows the average score of fluency and adequacy for 3 categories of sentences. Table 3.7[7] shows NE coverage in similar fashion.

[7]We kept Coverage of systems from section 2.5 and preprocessed experiments separate so as to not clutter the page

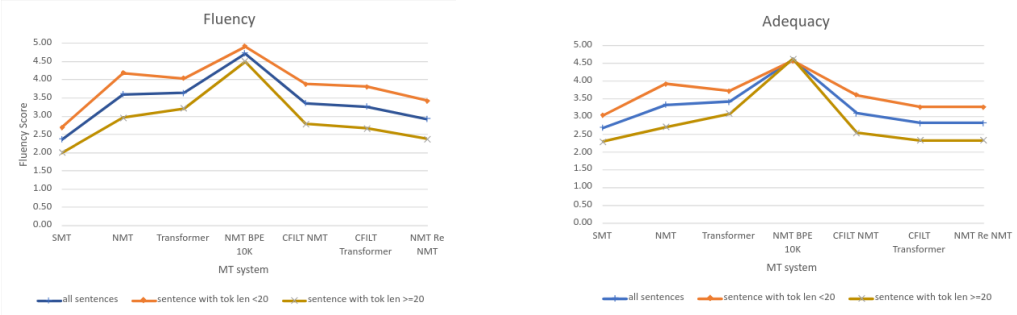


Figure 3.5: Average Fluency and Adequacy score for each translation systems with respect to input sentence length.

Sentences (# of NE)	SMT		NMT		Transformer	
	NE Trans	NE Not Trans	NE Trans	NE Not Trans	NE Trans	NE Not Trans
all sent (82 NE)	**74**	**8**	57	25	45	37
tok len <20 (21 NE)	**20**	**1**	18	3	13	8
tok len >= 20 (61 NE)	**54**	**7**	40	22	32	29

Table 3.6: NE Coverage for each system from Section 2.5

.

Sentences (# of NE)	NMT BPE 10K		CFILT NMT		CFILT Transformer		NMT Re Eng NMT	
	NE Trans	NE Not Trans	NE Trans	NE Not Trans	NE Trans	NE Not Trans	NE Trans	NE Not Trans
all sent (82 NE)	**79**	**3**	47	34	41	40	41	41
tok len <20 (21 NE)	**19**	**2**	14	6	11	10	12	9
tok len >= 20 (61 NE)	**60**	**1**	33	28	30	30	29	32

Table 3.7: NE coverage for Each system after preprocessing.

We can summarise our observations as follows,

- NMT with BPE had best NE coverage followed by SMT and then other neural methods in other words OOV issues for Neural models.
- Neural models have better fluency.
- Just reordering not good when compared with subword tokenized data. If reordering is erroneous the translation output will suffer.
- If we compare all the metrics (adequacy, fluency, NE coverage) we see those translations of longer input sentences suffer when compared with translations of shorted sentences for all systems other than NMT with BPE.

Now there is scope of improvement here in two fronts.

- Reordering: Improving NMT based reordering by using different algorithm and/or different neural network architecture (E.g transformer).
- Subword: Using subwords in reordering can alleviate some issues.

- Datasize: This was trained on 7.5 Lakh Parallel sentences. Increasing dataset size can help in translation quality [41].
- Segment and Translate: This also looks like a promising area, specially Kuang and Xiong [44], which does not require extensive linguistic resources other than parallel text. This would require exhaustive experimentation.

Now that we have covered how to create MT systems and what preprocessing techniques are used for language pairs with decent resources. Next section will look at techniques for low resource language pair. There has been some interesting work, which we believe can be useful in our context as well.

Chapter 4

Methods For Low resource Language Pair

We have seen basic formulation of SMT, NMT and transformer. We have also seen some extensions on these models as well as some preprocessing which have been done or could be done. These methodologies require a decent amount of parallel text for learning translation system. There is an interesting area of methods for low resource pair in MT which deals with solutions to build MT systems in case the amount of parallel text is low. There have been active shared tasks and workshops[1][2] solely focused on this area. For English $\iff$ Indian languages as we have unequal amount of parallel text, techniques in this area are vital. Some ways of dealing with low resourced pairs we will see in following subsections are as follows,

- 4.1 Using a 3rd language L_2 as pivot if L_1L_3 languages pair are low-resource. [99], [19]
- 4.2 Generating synthetic dataset for language pair L_1L_3 to increase bitext size. [85], [24], [2], [91]
- 4.3 Building NMT for high resource L_1L_2 and using that MT system as initialization for L_1L_3. [104]
- 4.4 Combining Multiple Language pairs and training 1 NMT system for all. [17], [33]
- 4.5 Making use of Monolingual dataset of L_1 and L_3 to train MT system.

4.1 Pivot Translation

As mentioned earlier, in pivot translation L_2 is used as pivot for L_1 and L_3. Now we will look at how this is modeled in SMT followed by how its modeled in NMT.

4.1.1 Pivoting in SMT

There are multiple methods for Pivoting SMT. For all we will consider L_1, L_3 as source and target respectively and L_2 as Pivot. Now, these methods are as follows,

Triangulation

Wu and Wang [99] modeled pivot by triangulation as populating phrase table for L_1, L_3. This was done by triangulating alignments between L_1L_2 and L_2L_3. This can increase the phrase table size exponentially, e.g. one phrase in L_1 has four phrases in L_2 which in turn has 3 phrases each in L_3, thereby creating 12 entries for L_1 as source.

[1] https://www.statmt.org/wmt21/similar.html
[2] https://sites.google.com/view/loresmt/

Pipeline/Transfer

de Gispert and Mariño [19] suggested using two SMT systems (English to Spanish and Spanish to Catalan) for English-Catalan with Spanish as pivot. Input sentence was translated to k Spanish sentences, which in turn were translated into Catalan sentences, and translation with higher score was chosen. Here there are two issues, first any error encountered in the first SMT system would be propagated. Second, this kind of multi-tier system will be slow.

Multiple Pivot languages with linear interpolation

Wu and Wang [99] also introduced linear interpolation build on triangulation. In this method, they combined different triangulations using weights. For L_1 and L_3 with set of pivot languages (L_2,L5), possible phrase tables would be from $L_1L_2L_3$ and L_1L5L_3. They assigned weights to each phrase table, with sum of weights being 1 and tunned to get better translation than just triangulation.

Creating Synthetic Bitext

de Gispert and Mariño [19] also suggested using Spanish-to-Catalan SMT system to translate Spanish text from English-Spanish bitext to Catalan and use resultant synthetic bitext for English-Catalan training. They found both creating synthetic bitext and Transfer pivoting produced comparable results. Utiyama and Isahara [96], Kunchukuttan et al. [48] found that triangulation is better approach than Pipeline in terms of performance. To address the problem of Dravidian to Indo Aryan language translation (lanugage pair taken was - Malayalam to Hindi) More et. al. [57] used combination of two techniques. Pivot by triangulation [99] was used to calculate phrase translation probability between Malayalam and Hindi (using Bengali, Gujarati, Konkani, Marathi, Punjabi, Tamil and Telugu as pivot one by one) and multiple decoding paths [40] was used to harness these multiple translation models. They also used Morfessor [86][3] to segment training text. They reported that Punjabi is best language among languages mentioned above to work as pivot when translating from Malayalam to Hindi. Building on More et. al. [57] Kunchukuttan et. al. [48] trained PBSMT models between source-pivot and pivot-target pairs using BPE/OS as basic unit. It was observed that the OS and BPE-level pivot models significantly outperform word, morpheme and character-level pivot models.

4.1.2 Pivoting in NMT

Now for using pivoting in NMT there is no straightforward notion of phrases. [9] made use of word embeddings as replacement. In NMT they modeled pivoting as how to bring $L_1L_2L_3$ closer in terms of word embeddings. To use L_2 as pivot between L_1 and L_3, this work made use of systems trained on (L_1, L_2), (L_2, L_3) and tries to bridge the word embedding between L_1, L_2 and L_3 using following training objectives,

- Embedding space between L_1L_2 and L_2L_3 for common words across languages should be same, so they penalize systems till it does not learn to do that.
- But that can be hard to accomplish. So, alternative training objective was to mitigate it by forming word embedding such that the distance between two embedding spaces is consistent for each common word across languages.
- By making use of small bitext between L_1 and L_3, they trained joint system such that sum of translation probability of sentences between L_1L_2 and L_2L_3 reaches optimal score.

4.2 Data Augmentation

In this section we will look at methods for generating synthetic bitext, which can be used to augment training dataset. While going through literature we found that there are two major categories in which techniques for data augmentation can be divided, namely, Back translation and generation via replacement.

[3]https://morfessor.readthedocs.io

Back translation

Sennrich et al. [85] trained NMT model on German English parallel text, used monolingual text to translate German Monolingual to English. This translated output along with German monolingual text was called synthetic parallel corpus. Then the system was further trained on this 'Back translated' parallel text. E.g. For English $\Longleftrightarrow$ Hindi, first train English to Hindi and Hindi to English systems(Figure 4.1).

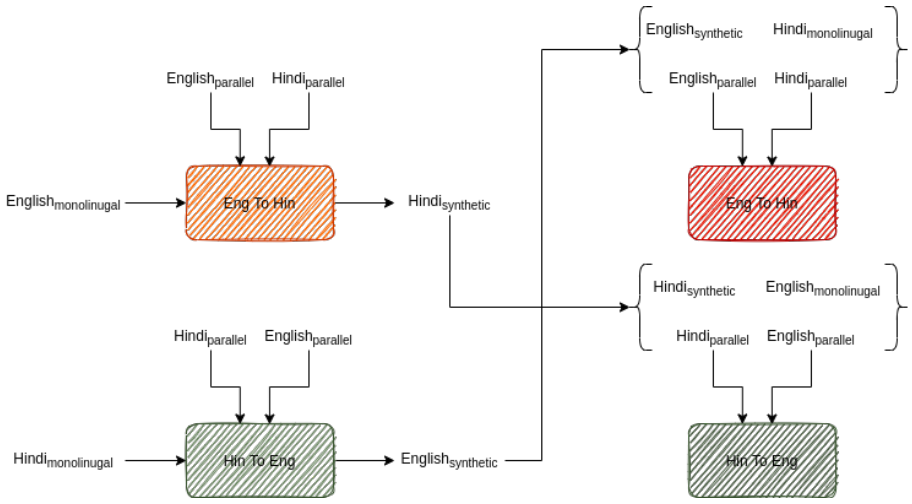


Figure 4.1: Flow of system training for data augmentation using Back translation

Followed by translation of $\text{Hindi}_{\text{Monolingual}}$ to generate $\text{English}_{\text{Synthetic}}$. And similarly generating $\text{Hindi}_{\text{Synthetic}}$ by translating $\text{English}_{\text{Monolingual}}$. Then in back translation strategy for data augmentation we will retrain English to Hindi by concatenating English-Hindi bitext with $\text{English}_{\text{Synthetic}}$-$\text{Hindi}_{\text{Monolingual}}$ bitext. Similarly Hindi to English system will be retrained using Hindi,$\text{Hindi}_{\text{Synthetic}}$ - English, $\text{English}_{\text{Monolingual}}$ bitext. Agrawal et al. [2] made use of back translation data from subpar MT model. Then trained on this synthetic dataset and fine-tuned on good parallel text. This whole operation was bootstrapped over some iterations till convergence. Dandapat and Federmann [18] made bootstrapped training on back translation to create Telugu $\Longleftrightarrow$ English System. Edunov et al. [23] observed that using back translation as data augmentation technique is as good as using human curated domain parallel text based on their observation of English German back translated data, which had low perplexity measured by Language Model when compared with samples from training set.

Similar to Back translation Imamura and Sumita [31] introduced forward translation. In this translation was done from source to target instead of target to source to prepare synthetic dataset. They only observed moderate improvements, but in contrast to backward translation, this approach does not require a pre-existing NMT system trained for the reverse translation direction.

Generation via replacement

In this type of data augmentation, we modify parallel text according to different methods and use the change text to augment training dataset.

Given source, target sentence pair <s,t> Fadaee et al. [24] altered it in a way that preserves the semantic equivalence between source and target while diversifying the training examples as much as possible. They replaced common words by a rare word in source and target, while keeping good fluency on both sides by using LM. Tennage et al. [91] used Parts of Speech (POS) tagging and morphological analysis as syntactic features to prune synthetic sentence pairs (Tamil, Sinhala), generated using , that do not adhere to respective language syntax. Song et al. [87] made use of dictionary to create a code-mix dataset and train. They did this to tackle the issue of NE translation. E.g. As we can see in 3rd row of Table 4.1 example, assuming there is a bilingual dictionary; we scan the input and find out a word W_s which existing in dictionary and use its dictionary entry W_t to switch the word in input, provided that target word W_t is also existing on the target side of training corpus. And augment the dataset with code switched sentence pairs.

	Source	Target
Input	I like to eat ice cream.	मुझे हिम मलाई खाना पसंद है।
Semantic Equivalence	I like to eat fruits.	मुझे फल खाना पसंद है।
Code switching	I like to eat हिम मलाई.	मुझे हिम मलाई खाना पसंद है।

Table 4.1: Different ways of generating synthetic dataset

Generating bitext from Embedding space

Rashid et al. [77] made an adversarial latent space based model [25] and tried to generate parallel sentences in two languages (English,French) concurrently and translating bidirectionally. Bilingual text generation goal is achieved by sampling from same latent space for both languages. Although no experiment was shown which can conclusively comment about quality of MT systems trained using these synthetically generated parallel texts.

Text Degeneration In all these methods, where there is text generation from Neural Models, there is a concern of text degradation as we have seen in some outputs in previous sections. This degradation is because text generation module of neural networks usually follows greedy or beam search approach to generate the next word. Holtzman et al. [30] tried to investigate why such phenomenon happens. They came to conclusion that likelihood maximizing decoding causes repetition and is overly generic language usage. Radford et al. [74] used top k sampling to make generated text little diverse. In top k sampling, word to be generated next is sampled from top k possibilities given previous context. It has a risk of sampling from the low-confidence tail of a model's predicted distribution. And hence they proposed nucleus (top p) sampling which uses probability distribution to to determine sets of tokens to be sampled from (it tries to find smallest set of tokens which can come next with total cumulative probability of p).

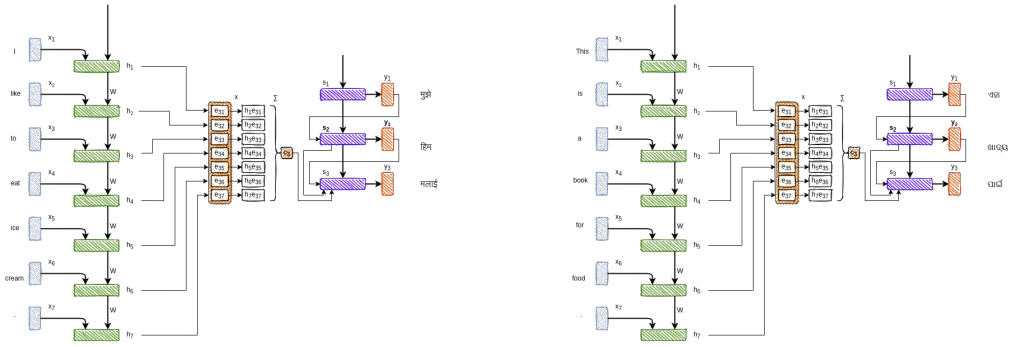
4.3 Transfer learning

Zoph et al. [104] trained an NMT based translation model (called parent model) for French→English translation. This trained model was then used the learned model as initializing weights in NMT training to build translation system (called child model) for resource poor language pair (Uzbek, English). After copying the weights from parent model to child model, parameters of English embeddings are frozen and Uzbek words are mapped to random French embeddings, thereafter parameters are fine-tuned while keeping English embeddings as they are. In low resource setting, their intuition for this kind of learning is that a strong prior distribution over models will be helpful. The parent model trained on a large amount of bilingual data can be considered an anchor point, the peak of our prior distribution in model space. When they trained the child model initialized with the parent model, they fixed parameters likely to be useful across tasks so that these parameters will not be changed during child model training. They reported increased BLEU Uzbek to English translation.

E.g. in Figure 4.2 we have pictorial representation of Transfer learning from English→Hindi to English→Odia. After training English→Hindi, one would have a network already pretrained at least on the encoder side for English. Now using these parameters from English to Hindi system we would train English to Odia, and since both these languages Hindi, Odia shares some commonality among them, Odia in this example can benefit from this kind of system.

Using transfer learning Zoph et al. [104], Dabre et al. [15] explored systems trained, using Hindi, Indonesian, Turkish, Russian, German and French as parent languages with Luxembourgish, Hausa, Somali, Marathi, Punjabi, Malayalam, Uzbek, Javanese, Kazakah and Sundanese as child languages while target language was English. And found that child systems performed best when the source of parent systems belonged to same language family.

Nguyen and Chiang [60] used Zoph et al. [104]'s idea in case source language in parent model is also low-resource but related source language in child model (Turkish, Uzbek, and Uyghur to English language pair). They found that transfer learning [104] does not work in this setting. Their introduction of joint BPE (finding BPE for both parent and child language together) increased the performance of translation systems a bit.



(a) Parent System (b) Child System

Figure 4.2: Example of transfer learning.

4.4 Multilingual Neural Machine Translation

A progression from Transfer learning was use of both *Parent*[4] language pair and *Child* language pair together for training, and then fine tune them according to the language pair of choice. This could help low resource language pair to be trained better. This is training setting for Multilingual Machine Translation [17], where one system tries to learn parameters for translating ideally from many to many languages.

Using a shared wordpiece [100] vocabulary and network parameters of standard NMT system, Johnson et al. [33]'s approach used a single model to train many to many translation system. To indicate target language an artificial token is inserted at the start of input sentence. E.g. when translating from English to Hindi *<2hi>* would be inserted. They used shared parameters to force the model to generalize across language boundaries during training. They found the translation scores for low resource language pair improved. Also, for English Centric NMT (English to Multiple languages and vice versa), the scores for Multiple Languages to English was better than one-to-one systems. But scores of English to Multiple languages and Many to Many languages were less then one-to-one systems. This does point to fact that decoder was well attuned to generating English in all Multiple Languages to English cases. Which was not the case for English to Multiple Languages, model was trying to learn multiple languages. This kind of Multilingual system requires less number of parameters to train when compared with total parameters to be trained in one-to-one scenario. Also, since this system used multiple languages, it was able to handle code-switching of terms in between. And it also gave rise to an interesting idea based on premise of shared parameters, *zero-shot translation* between two languages X and Y. Say there is no parallel corpus between X and Y, but there is parallel text between X,Z and W,Y. So, in this scenario if there was a language identification token of *<toY>*, system produces some output. Though translation might not be optimal translation.

Lakew et al. [50] looked at the question about which one is better for multilingual, transformer or sequential networks; for this setting, they found that transformer models work better when compared with RNN. This experimentation was done on Dutch Romanian to German and Italian. They also found that most of the errors on recurrent and transform network were related to lexical errors, so the translation of word was a problematic rather than morphological Inflexion or reordering.

Aharoni et al. [3] trained an English centric many to many models where English was either on source or target. This was done to check how many languages together can be used for Multilingual system such that translation quality over all is comparable to each 1 to 1 baseline system. They made use of 109 languages. Of which 58 were low resource which had bitext ranging from 3.2K sentence pair to 214K. In multilingual setting, for translation to English, many to many systems fared better than many to one. For English to all other languages one to many fared better than many to many. Now for high resource languages, where average sentence pair were 940K, for X to English, many to one fared better. For Eng to X one to many fared better. What also was seen here was the effect of addition of new languages in multilingual system. As they went up the number of languages shared to train a system, the BLEU score suffered negatively. And they also

[4]When we say parent and child language pair in this context it means the language pairs if they would have been used in transfer learning and not in the sense of one language is derived from another.

looked at how many languages would build a system with optimal zero shot translations. And they found that when 50 languages were involved in training the system, zero shot translation was optimal.

Then Hokamp et al. [29] did a study regarding sharing of resources in Multilingual Scenario. Their study was to find sharing which resources and keeping which parameters separate can increase the performance of multilingual systems. Different types of decoders were used for transformers,

- Shared decoder with language token to disambiguate target languages.
- There was a decoder which had unique word embeddings for each language, but decoder was shared.
- There was a decoder for each language.
- There was a decoder which was shared but the self-attention networks were different for each language.

They found that out of 20, for 17 language directions either a unique decoder or unique attention mechanism helped.

Dabre et al. [16] looked at how low resource languages fair with high resource language in multilingual setting. They found that to get better results there is a need to fine tune multilingual model on small low resource bitext.

Tan et al. [90] worked on how the languages should be grouped for multilingual training, i.e., which all languages should be together to get a optimal multilingual translation system. This was done for 23 languages with English centric system in mind. They used two methods to group languages,

- Prior knowledge: languages were grouped based on their respective language families.
- Language embedding: They trained 23 languages with a multilingual NMT system. And they looked at embeddings of resultant language identification tags, used to identify which language to translate to. Embeddings of these tags were used for hierarchical clustering to find out which language should be used together.

They found that systems trained with language embedding gave a better result than using prior knowledge.

Rubino et al. [79] looked at different techniques for MT in very low resource environment. They made use of multiple data augmentation techniques like back translation, forwards translation, unsupervised NMT and unsupervised SMT (more on unsupervised MT in subsection 4.5). They found that extremely low resource and MT requires large background data to compete with SMT approach, so SMT does work well in a resource scares environment. But using multilingual NMT, NMT can reach somewhere near SMT environment.

4.5 Unsupervised Machine Translation

Till now we have seen techniques which make use of parallel text. But there exist language pairs with very less or no parallel text dataset. For that, Lample et al. [51] proposed a model that takes sentences from monolingual corpora in two different languages and maps them into the same latent space. They did this in stages (Figure 4.3),

A Find the embedding space for each monolingual corpus.

B Initialization: Two distributions are roughly aligned using using Conneau et al. [13] to make a parallel dictionary from two monolingual texts.

C Language Modeling: Learn LM on respective monolingual dataset, this acted as a data-driven prior to correct sentences.

D Back-translation: Using observed source sentence, they translated (red dash arrow) to target sentence, yielding noisy translation. From here they back - translated (blue arrow) in source language to give a reconstructed source sentence. And this discrepancy between original and reconstructed source sentence provided the error signal for training target to source model parameters. Same was done in reverse direction to train source to target model.

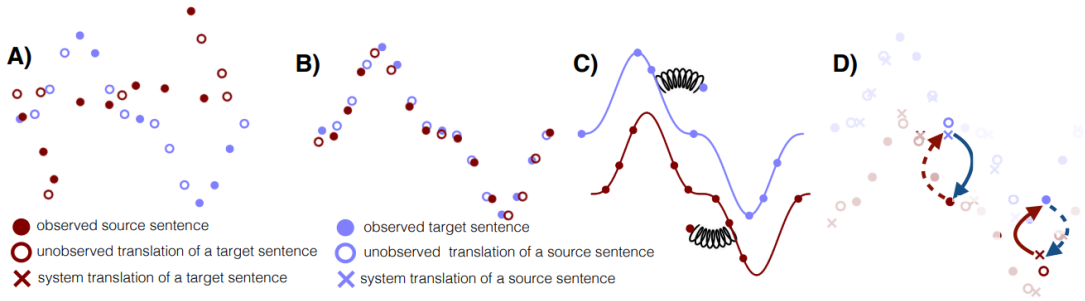


Figure 4.3: Toy illustration of the three principles of unsupervised MT

Similarly for Unsupervised NMT, Lample et al. [52] made use of model to translate a sentence and use PBSMT to extract phrases, do back-translation and continue doing this in iterative fashion.

But Kim et al. [34] found that for unsupervised MT to produce decent translation monolingual text on both source and target should belong to same domain and shouldn't be too distant, otherwise performance is severely affected by linguistic dissimilarity and domain mismatch between source and target monolingual data.

Chapter 5

Going Forward

In this report we have seen various methods to build MT systems in general and for Indian languages. We also looked at various preprocessing steps like tokenization using sub-words, reordering and splitting followed by translation. We have also seen methods specifically designed for low resource language pair, like data augmentation using back translation, code switching and semantic equivalence replacement. We have seen different methods for training machine translation systems in a low resource setting.

We took part in various shared tasks using one or two concepts mentioned in this report to see how well these techniques would work compared to other methods employed by other researchers. In Yadav et al. [103], we saw statistical methods working better than neural methods for language pair in *ENIL* and low resource setting for extremely low resource langauges like English to Maghai. SMT system with preprocessing sub-words was best for *ENIL* setting. In Yadav and Shrivastava [101] we looked at similar languages – Hindi and Marathi. We trained using both NMT and SMT using preprocessing steps like sub words and date augmentation technique like back translation. Our Marathi to Hindi SMT system was in top five systems.

In Yadav and Shrivastava [102], we build English to Marathi systems using reordering, sub words and back translation using SMT. And our system was in top 3 systems under constrained conditions[1]. With our analysis of systems developed for these shared tasks and after looking at the experiments we have done in *ENIL* settings, we have planned future research experiments in the following manner,

- Languages: We are building translation systems for English to {Hindi, Odia, Malayalam and Telugu}. These languages are chosen so that we can look at methods and techniques from the following point of view,
  - Amount of data set: all these language pairs have bitext with varying degree of size available. Which poses an interesting challenge in terms of data preprocessing and system training.
  - Nature of languages: Hindi and Odia come under Indo-Aryan language family whereas Malayalam and Telugu come under Dravidian family. Which is another interesting challenge how to make use of commonality and differences between these languages to decide which methods and preprocessing steps are optimal for training.
- Experiments concerning preprocessing: As we have seen in various literature [47] [85], experiments and shared task, data driven methods can only go so far without analysing bitext, methods to build amiable systems for language pairs. We will be looking at different kinds of preprocessing for each language and how these preprocessing steps perform for each language pair and architecture.
  - Subwords: We will looking at the extent to which subwords can help in this regard.
  - Reordering: We have seen that translation systems making use of SMT have a good coverage of named entities, but in *ENIL* setting due to difference in syntax being too much, they can produce erroneous outputs, reordering English has been employed sucessfully

[1] Making use of dataset provided by shared task organizers only to build systems

to allievate this. So. we will be also looking at reordering as another processing step. There are different ways of reordering as mentioned in the section. We will be looking at some of them for each language pair and see how systems trained with reordering can affect translation output.

  - Divide and translate: And we will further, like to analyse longer sentences in English and their translations using respective machine translation engines and see if there is a scope for splitting the input and translate it in parts.

- Domain adaptation and named entity coverage are some of the important areas in machine translation and if we take *ENIL* setting into account, their importance cannot be Understated. We will be looking at data augmentation techniques like Back translation [85], Code switching[87] and Semantic equivalence replacement [24] for this. This set of experiments would be to investigate which of these methods or whether combinations of these methods can aid in building good translation systems.

- Now when we say good translation or when we are talking about evaluation of a translation systems, two important things are - test data set and evaluation methods used for evaluating each system on said test dataset.

  - Test data set: ideally should be representative enough of every aspect of source language in terms of kind of sentences in source language with the respective golden translation in target language. Which can help in evaluating each MT system. In this regard, since we will be dealing with multiple languages pairs, we are going to use WAT2021 Indic-Multilingual MT test data set[2]. This is to ensure that if we extend our target language pool, we will have a corresponding test set available.
  - Evaluation methods: as we have already discussed earlier in Section 2.3 there are multiple ways of evaluating translation system. For this work, we plan to do two-fold analysis for translation systems.
    * Automatic: we would be evaluating using BLEU scores using sacrebleu [71]. Other than that we will also look at chrF [70] scores. We are also interested in looking at named entity coverage as one of the elements for machine translation performance, to see how well the named entities are being handled by systems.
    * Manual: we also plan to manually analyse (at least for English to Hindi), the translations from different sentences on a representative set of input sentences, while making sure that these inputs are of different types and length. So that we have a clearer picture in terms of how systems perform in different scenario.

With this work we hope to investigate the flow of preprocessing steps and techniques which can be used to train a system in respective language pairs. While also looking at different aspects of translation performance to judge a system.

[2] http://lotus.kuee.kyoto-u.ac.jp/WAT/indic-multilingual/

Bibliography

[1] Herve´ Abdi. Partial least squares (pls) regression. In *In Encyclopedia of Social Sciences Research*. Sage Publications.

[2] Ruchit Agrawal, Mihir Shekhar, and Dipti Sharma. Three-phase training to address data sparsity in neural machine translation. In *Proceedings of the 14th International Conference on Natural Language Processing (ICON-2017)*, pages 13–22, Kolkata, India, December 2017. NLP Association of India. URL https://www.aclweb.org/anthology/W17-7503.

[3] Roee Aharoni, Melvin Johnson, and Orhan Firat. Massively multilingual neural machine translation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 3874–3884, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1388. URL https://aclanthology.org/N19-1388.

[4] A. V. Aho and J. D. Ullman. Syntax directed translations and the pushdown assembler. *J. Comput. Syst. Sci.*, 3(1):37–56, February 1969. ISSN 0022-0000. doi: 10.1016/S0022-0000(69)80006-1. URL http://dx.doi.org/10.1016/S0022-0000(69)80006-1.

[5] Gary Anthes. Automated translation of indian languages. *Commun. ACM*, 53(1):24–26, January 2010. ISSN 0001-0782. doi: 10.1145/1629175.1629184. URL http://doi.acm.org/10.1145/1629175.1629184.

[6] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.

[7] Pushpak Bhattacharyya, Mitesh M. Khapra, and Anoop Kunchukuttan. Statistical machine translation between related languages. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Tutorial Abstracts*, pages 17–20, San Diego, California, June 2016. Association for Computational Linguistics. doi: 10.18653/v1/N16-4006. URL https://www.aclweb.org/anthology/N16-4006.

[8] Peter F Brown, Vincent J Della Pietra, Stephen A Della Pietra, and Robert L Mercer. The mathematics of statistical machine translation: Parameter estimation. *Computational linguistics*, 19(2):263–311, 1993.

[9] Yong Cheng. Joint training for pivot-based neural machine translation. In *Joint Training for Neural Machine Translation*, pages 41–54. Springer, 2019.

[10] David Chiang. Hierarchical phrase-based translation. *Computational Linguistics*, 33 (2):201–228, 2007. doi: 10.1162/coli.2007.33.2.201. URL https://aclanthology.org/J07-2003.

[11] Kyunghyun Cho, Bart van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. On the properties of neural machine translation: Encoder–decoder approaches. In *Proceedings of SSST-8, Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation*, pages 103–111, Doha, Qatar, October 2014. Association for Computational Linguistics. doi: 10.3115/v1/W14-4012. URL https://aclanthology.org/W14-4012.

[12] Michael Collins, Philipp Koehn, and Ivona Kučerová. Clause restructuring for statistical machine translation. In *Proceedings of the 43rd Annual Meeting of the Association for Computational Linguistics (ACL'05)*, pages 531–540, Ann Arbor, Michigan, June

2005. Association for Computational Linguistics. doi: 10.3115/1219840.1219906. URL https://aclanthology.org/P05-1066.

[13] Alexis Conneau, Guillaume Lample, Marc'Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Word translation without parallel data. *CoRR*, abs/1710.04087, 2017. URL http://arxiv.org/abs/1710.04087.

[14] Raj Dabre, Jyotesh Choudhari, and Pushpak Bhattacharyya. Tackling close cousins: Experiences in developing statistical machine translation systems for Marathi and Hindi. In *Proceedings of the 11th International Conference on Natural Language Processing*, pages 11–19, Goa, India, December 2014. NLP Association of India. URL https://www.aclweb.org/anthology/W14-5103.

[15] Raj Dabre, Tetsuji Nakagawa, and Hideto Kazawa. An empirical study of language relatedness for transfer learning in neural machine translation. In *Proceedings of the 31st Pacific Asia Conference on Language, Information and Computation*, pages 282–286. The National University (Phillippines), November 2017. URL https://www.aclweb.org/anthology/Y17-1038.

[16] Raj Dabre, Atsushi Fujita, and Chenhui Chu. Exploiting multilingualism through multistage fine-tuning for low-resource neural machine translation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 1410–1416, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1146. URL https://aclanthology.org/D19-1146.

[17] Raj Dabre, Chenhui Chu, and Anoop Kunchukuttan. A survey of multilingual neural machine translation. *ACM Comput. Surv.*, 53(5), September 2020. ISSN 0360-0300. doi: 10.1145/3406095. URL https://doi.org/10.1145/3406095.

[18] Sandipan Dandapat and Christian Federmann. Iterative data augmentation for neural machine translation: a low resource case study for english–telugu, 2018.

[19] Adrià de Gispert and José B Mariño. Catalan-english statistical machine translation without parallel corpus: Bridging through spanish. *Strategies for developing machine translation for minority languages*, page 65.

[20] John DeNero and Jakob Uszkoreit. Inducing sentence structure from parallel corpora for reordering. In *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing*, pages 193–203, Edinburgh, Scotland, UK., July 2011. Association for Computational Linguistics. URL https://aclanthology.org/D11-1018.

[21] Pratik Desai, Amit Sangodkar, and Om P. Damani. A domain-restricted, rule based, English-Hindi machine translation system based on dependency parsing. In *Proceedings of the 11th International Conference on Natural Language Processing*, pages 177–185, Goa, India, December 2014. NLP Association of India. URL https://www.aclweb.org/anthology/W14-5128.

[22] Shuoyang Ding, Adithya Renduchintala, and Kevin Duh. A call for prudent choice of subword merge operations in neural machine translation. In *Proceedings of Machine Translation Summit XVII: Research Track*, pages 204–213, Dublin, Ireland, August 2019. European Association for Machine Translation. URL https://aclanthology.org/W19-6620.

[23] Sergey Edunov, Myle Ott, Michael Auli, and David Grangier. Understanding back-translation at scale. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 489–500, Brussels, Belgium, October-November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1045. URL https://www.aclweb.org/anthology/D18-1045.

[24] Marzieh Fadaee, Arianna Bisazza, and Christof Monz. Data augmentation for low-resource neural machine translation. *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2017. doi: 10.18653/v1/p17-2090. URL http://dx.doi.org/10.18653/v1/P17-2090.

[25] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014.

[26] Jiatao Gu, Zhengdong Lu, Hang Li, and Victor O.K. Li. Incorporating copying mechanism in sequence-to-sequence learning. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1631–1640, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1154. URL https://aclanthology.org/P16-1154.

[27] Kenneth Heafield. KenLM: Faster and smaller language model queries. In *Proceedings of the Sixth Workshop on Statistical Machine Translation*, pages 187–197, Edinburgh, Scotland, July 2011. Association for Computational Linguistics. URL https://aclanthology.org/W11-2123.

[28] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9 (8):1735–1780, 1997.

[29] Chris Hokamp, John Glover, and Demian Gholipour Ghalandari. Evaluating the supervised and zero-shot performance of multi-lingual translation models. In *Proceedings of the Fourth Conference on Machine Translation (Volume 2: Shared Task Papers, Day 1)*, pages 209–217, Florence, Italy, August 2019. Association for Computational Linguistics. doi: 10.18653/v1/W19-5319. URL https://aclanthology.org/W19-5319.

[30] Ari Holtzman, Jan Buys, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. *CoRR*, abs/1904.09751, 2019. URL http://arxiv.org/abs/1904.09751.

[31] Kenji Imamura and Eiichiro Sumita. NICT self-training approach to neural machine translation at NMT-2018. In *Proceedings of the 2nd Workshop on Neural Machine Translation and Generation*, pages 110–115, Melbourne, Australia, July 2018. Association for Computational Linguistics. doi: 10.18653/v1/W18-2713. URL https://aclanthology.org/W18-2713.

[32] Shonosuke Ishiwatari, Jingtao Yao, Shujie Liu, Mu Li, Ming Zhou, Naoki Yoshinaga, Masaru Kitsuregawa, and Weijia Jia. Chunk-based decoder for neural machine translation. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1901–1912, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1174. URL https://aclanthology.org/P17-1174.

[33] Melvin Johnson, Mike Schuster, Quoc V. Le, Maxim Krikun, Yonghui Wu, Zhifeng Chen, Nikhil Thorat, Fernanda Viégas, Martin Wattenberg, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google's multilingual neural machine translation system: Enabling zero-shot translation. *Transactions of the Association for Computational Linguistics*, 5:339–351, 2017. doi: 10.1162/tacl_a_00065. URL https://www.aclweb.org/anthology/Q17-1024.

[34] Yunsu Kim, Miguel Graça, and Hermann Ney. When and why is unsupervised neural machine translation useless? In *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation*, pages 35–44, Lisboa, Portugal, November 2020. European Association for Machine Translation. URL https://aclanthology.org/2020.eamt-1.5.

[35] Dan Klein and Christopher D. Manning. Accurate unlexicalized parsing. In *Proceedings of the 41st Annual Meeting of the Association for Computational Linguistics*, pages 423–430, Sapporo, Japan, July 2003. Association for Computational Linguistics. doi: 10.3115/1075096.1075150. URL https://www.aclweb.org/anthology/P03-1054.

[36] Guillaume Klein, Yoon Kim, Yuntian Deng, Vincent Nguyen, Jean Senellart, and Alexander Rush. OpenNMT: Neural machine translation toolkit. In *Proceedings of the 13th Conference of the Association for Machine Translation in the Americas (Volume 1: Research Track)*, pages 177–184, Boston, MA, March 2018. Association for Machine Translation in the Americas. URL https://aclanthology.org/W18-1817.

[37] Philipp Koehn. *Statistical Machine Translation*. Cambridge University Press.

[38] Philipp Koehn. *Noun Phrase Translation*. PhD thesis, Los Angeles, CA, USA, 2003. AAI3133297.

[39] Philipp Koehn. Pharaoh: a beam search decoder for phrase-based statistical machine translation models. In *Conference of the Association for Machine Translation in the Americas*, pages 115–124. Springer, 2004.

[40] Philipp Koehn and Hieu Hoang. Factored translation models. In *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, pages 868–876, Prague, Czech Republic, June 2007. Association for Computational Linguistics. URL https://www.aclweb.org/anthology/D07-1091.

[41] Philipp Koehn and Rebecca Knowles. Six challenges for neural machine translation. In *Proceedings of the First Workshop on Neural Machine Translation*, pages 28–39, Vancouver, August 2017. Association for Computational Linguistics. doi: 10.18653/v1/W17-3204. URL https://www.aclweb.org/anthology/W17-3204.

[42] Philipp Koehn, Franz Josef Och, and Daniel Marcu. Statistical phrase based translation. In *Proceedings of the 2003 Conference of the North American Chapter of the Association for Computational Linguistics on Human Language Technology-Volume 1*, pages 48–54. Association for Computational Linguistics, 2003.

[43] Philipp Koehn, Hieu Hoang, Alexandra Birch, Chris Callison-Burch, Marcello Federico, Nicola Bertoldi, Brooke Cowan, Wade Shen, Christine Moran, Richard Zens, et al. Moses: Open source toolkit for statistical machine translation. In *Proceedings of the 45th annual meeting of the association for computational linguistics companion volume proceedings of the demo and poster sessions*, pages 177–180, 2007.

[44] Shaohui Kuang and Deyi Xiong. Automatic long sentence segmentation for neural machine translation. In Chin-Yew Lin, Nianwen Xue, Dongyan Zhao, Xuanjing Huang, and Yansong Feng, editors, *Natural Language Understanding and Intelligent Applications*, pages 162–174, Cham, 2016. Springer International Publishing. ISBN 978-3-319-50496-4.

[45] Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 66–75, Melbourne, Australia, July 2018. Association for Computational Linguistics. doi: 10.18653/v1/P18-1007. URL https://aclanthology.org/P18-1007.

[46] Anoop Kunchukuttan and Pushpak Bhattacharyya. Orthographic syllable as basic unit for SMT between related languages. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1912–1917, Austin, Texas, November 2016. Association for Computational Linguistics. doi: 10.18653/v1/D16-1196. URL https://aclanthology.org/D16-1196.

[47] Anoop Kunchukuttan, Abhijit Mishra, Rajen Chatterjee, Ritesh M. Shah, and Pushpak Bhattacharyya. Shata-anuvadak: Tackling multiway translation of indian languages. In *LREC*, 2014.

[48] Anoop Kunchukuttan, Maulik Shah, Pradyot Prakash, and Pushpak Bhattacharyya. Utilizing lexical similarity between related, low-resource languages for pivot-based SMT. In *Proceedings of the Eighth International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, pages 283–289, Taipei, Taiwan, November 2017. Asian Federation of Natural Language Processing. URL https://www.aclweb.org/anthology/I17-2048.

[49] Anoop Kunchukuttan, Pratik Mehta, and Pushpak Bhattacharyya. The IIT Bombay English-Hindi parallel corpus. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*, Miyazaki, Japan, May 2018. European Language Resources Association (ELRA). URL https://aclanthology.org/L18-1548.

[50] Surafel Melaku Lakew, Mauro Cettolo, and Marcello Federico. A comparison of transformer and recurrent neural networks on multilingual neural machine translation. In *Proceedings of the 27th International Conference on Computational Linguistics*, pages 641–652, Santa Fe, New Mexico, USA, August 2018. Association for Computational Linguistics. URL https://aclanthology.org/C18-1054.

[51] Guillaume Lample, Alexis Conneau, Ludovic Denoyer, and Marc'Aurelio Ranzato. Unsupervised machine translation using monolingual corpora only, 2017.

[52] Guillaume Lample, Myle Ott, Alexis Conneau, Ludovic Denoyer, and Marc'Aurelio Ranzato. Phrase-based & neural unsupervised machine translation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 5039–5049, Brussels, Belgium, October-November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1549. URL https://aclanthology.org/D18-1549.

[53] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130*, 2017.

[54] Thang Luong, Hieu Pham, and Christopher D. Manning. Effective approaches to attention-based neural machine translation. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1412–1421, Lisbon, Portugal, September 2015. Association for Computational Linguistics. doi: 10.18653/v1/D15-1166. URL https://aclanthology.org/D15-1166.

[55] Lesly Miculicich, Dhananjay Ram, Nikolaos Pappas, and James Henderson. Document-level neural machine translation with hierarchical attention networks. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2947–2954, Brussels, Belgium, October-November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1325. URL https://aclanthology.org/D18-1325.

[56] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[57] Rohit More, Anoop Kunchukuttan, Pushpak Bhattacharyya, and Raj Dabre. Augmenting pivot based SMT with word segmentation. In *Proceedings of the 12th International Conference on Natural Language Processing*, pages 303–307, Trivandrum, India, December 2015. NLP Association of India. URL https://www.aclweb.org/anthology/W15-5944.

[58] Tetsuji Nakagawa. Efficient top-down BTG parsing for machine translation preordering. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 208–218, Beijing, China, July 2015. Association for Computational Linguistics. doi: 10.3115/v1/P15-1021. URL https://aclanthology.org/P15-1021.

[59] Graham Neubig, Taro Watanabe, and Shinsuke Mori. Inducing a discriminative parser to optimize machine translation reordering. In *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, pages 843–853, Jeju Island, Korea, July 2012. Association for Computational Linguistics. URL https://aclanthology.org/D12-1077.

[60] Toan Q. Nguyen and David Chiang. Transfer learning across low-resource, related languages for neural machine translation. In *Proceedings of the Eighth International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, pages 296–301, Taipei, Taiwan, November 2017. Asian Federation of Natural Language Processing. URL https://www.aclweb.org/anthology/I17-2050.

[61] JOAKIM NIVRE, JOHAN HALL, JENS NILSSON, ATANAS CHANEV, GÜLŞEN ERYİGİT, SANDRA KÜBLER, SVETOSLAV MARINOV, and ERWIN MARSI. Maltparser: A language-independent system for data-driven dependency parsing. *Natural Language Engineering*, 13(2):95–135, 2007. doi: 10.1017/S1351324906004505.

[62] Franz Josef Och. Minimum error rate training in statistical machine translation. In *Proceedings of the 41st Annual Meeting on Association for Computational Linguistics-Volume 1*, pages 160–167. Association for Computational Linguistics, 2003.

[63] Franz Josef Och and Hermann Ney. A systematic comparison of various statistical alignment models. *Computational linguistics*, 29(1):19–51, 2003.

[64] Franz Josef Och and Hermann Ney. The alignment template approach to statistical machine translation. *Comput. Linguist.*, 30(4):417–449, December 2004. ISSN 0891-2017. doi: 10.1162/0891201042544884. URL http://dx.doi.org/10.1162/0891201042544884.

[65] Franz Josef Och, Nicola Ueffing, and Hermann Ney. An efficient a* search algorithm for statistical machine translation. In *Proceedings of the workshop on Data-driven methods in machine translation-Volume 14*, pages 1–8. Association for Computational Linguistics, 2001.

[66] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting on association for computational linguistics*, pages 311–318. Association for Computational Linguistics, 2002.

[67] Raj Nath Patel, Rohit Gupta, Prakash B. Pimpale, and Sasikumar M. Reordering rules for English-Hindi SMT. In *Proceedings of the Second Workshop on Hybrid Approaches to Translation*, pages 34–41, Sofia, Bulgaria, August 2013. Association for Computational Linguistics. URL https://www.aclweb.org/anthology/W13-2807.

[68] Jeffrey Pennington, Richard Socher, and Christopher Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014.

[69] Prakash B. Pimpale, Raj Nath Patel, and Sasikumar M. SMT from agglutinative languages: Use of suffix separation and word splitting. In *Proceedings of the 11th International Conference on Natural Language Processing*, pages 2–10, Goa, India, December 2014. NLP Association of India. URL https://www.aclweb.org/anthology/W14-5102.

[70] Maja Popović. chrF: character n-gram F-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, pages 392–395, Lisbon, Portugal, September 2015. Association for Computational Linguistics. doi: 10.18653/v1/W15-3049. URL https://aclanthology.org/W15-3049.

[71] Matt Post. A call for clarity in reporting BLEU scores. In *Proceedings of the Third Conference on Machine Translation: Research Papers*, pages 186–191, Brussels, Belgium, October 2018. Association for Computational Linguistics. doi: 10.18653/v1/W18-6319. URL https://aclanthology.org/W18-6319.

[72] Jean Pouget-Abadie, Dzmitry Bahdanau, Bart van Merriënboer, Kyunghyun Cho, and Yoshua Bengio. Overcoming the curse of sentence length for neural machine translation using automatic segmentation. In *Proceedings of SSST-8, Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation*, pages 78–85, Doha, Qatar, October 2014. Association for Computational Linguistics. doi: 10.3115/v1/W14-4009. URL https://aclanthology.org/W14-4009.

[73] Ye Qi, Devendra Sachan, Matthieu Felix, Sarguna Padmanabhan, and Graham Neubig. When and why are pre-trained word embeddings useful for neural machine translation? In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 529–535, New Orleans, Louisiana, June 2018. Association for Computational Linguistics. doi: 10.18653/v1/N18-2084. URL https://aclanthology.org/N18-2084.

[74] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.

[75] Ananthakrishnan Ramanathan, Jayprasad Hegde, Ritesh M. Shah, Pushpak Bhattacharyya, and Sasikumar M. Simple syntactic and morphological processing can help English-Hindi statistical machine translation. In *Proceedings of the Third International Joint Conference on Natural Language Processing: Volume-I*, 2008. URL https://www.aclweb.org/anthology/I08-1067.

[76] Gowtham Ramesh, Sumanth Doddapaneni, Aravinth Bheemaraj, Mayank Jobanputra, Raghavan AK, Ajitesh Sharma, Sujit Sahoo, Harshita Diddee, Mahalakshmi J, Divyanshu Kakwani, Navneet Kumar, Aswin Pradeep, Kumar Deepak, Vivek Raghavan, Anoop Kunchukuttan, Pratyush Kumar, and Mitesh Shantadevi Khapra. Samanantar: The largest publicly available parallel corpora collection for 11 indic languages, 2021.

[77] Ahmad Rashid, Alan Do-Omri, Md. Akmal Haidar, Qun Liu, and Mehdi Rezagholizadeh. Bilingual-GAN: A step towards parallel text generation. In *Proceedings of the Workshop on Methods for Optimizing and Evaluating Neural Language Generation*, pages 55–64, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/W19-2307. URL https://aclanthology.org/W19-2307.

[78] Xin Rong. word2vec parameter learning explained. *arXiv preprint arXiv:1411.2738*, 2014.

[79] Raphael Rubino, Benjamin Marie, Raj Dabre, Atushi Fujita, Masao Utiyama, and Eiichiro Sumita. Extremely low-resource neural machine translation for asian languages. *Machine Translation*, 34(4):347–382, 2020.

[80] Kunal Sachdeva and Dipti Sharma. Exploring the effect of semantic similarity for phrase-based machine translation. In *Proceedings of the 3rd Workshop on Continuous Vector Space Models and their Compositionality*, pages 41–47, Beijing, China, July 2015. Association for Computational Linguistics. doi: 10.18653/v1/W15-4005. URL https://www.aclweb.org/anthology/W15-4005.

[81] Kunal Sachdeva, Rishabh Srivastava, Sambhav Jain, and Dipti Sharma. Hindi to English machine translation: Using effective selection in multi-model SMT. In *Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC'14)*, pages 1807–1811, Reykjavik, Iceland, May 2014. European Language Resources Association (ELRA). URL http://www.lrec-conf.org/proceedings/lrec2014/pdf/682_Paper.pdf.

[82] Rico Sennrich and Barry Haddow. Linguistic input features improve neural machine translation. In *Proceedings of the First Conference on Machine Translation: Volume 1, Research Papers*, pages 83–91, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/W16-2209. URL https://aclanthology.org/W16-2209.

[83] Rico Sennrich and Biao Zhang. Revisiting low-resource neural machine translation: A case study. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 211–221, 2019.

[84] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1162. URL https://www.aclweb.org/anthology/P16-1162.

[85] Rico Sennrich, Barry Haddow, and Alexandra Birch. Improving neural machine translation models with monolingual data. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 86–96, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1009. URL https://aclanthology.org/P16-1009.

[86] Peter Smit, Sami Virpioja, Stig-Arne Grönroos, and Mikko Kurimo. Morfessor 2.0: Toolkit for statistical morphological segmentation. In *Proceedings of the Demonstrations at the 14th Conference of the European Chapter of the Association for Computational Linguistics*, pages 21–24, Gothenburg, Sweden, April 2014. Association for Computational Linguistics. doi: 10.3115/v1/E14-2006. URL https://www.aclweb.org/anthology/E14-2006.

[87] Kai Song, Yue Zhang, Heng Yu, Weihua Luo, Kun Wang, and Min Zhang. Code-switching for enhancing NMT with pre-specified translation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 449–459, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1044. URL https://aclanthology.org/N19-1044.

[88] Katsuhito Sudoh, Kevin Duh, Hajime Tsukada, Tsutomu Hirao, and Masaaki Nagata. Divide and translate: Improving long distance reordering in statistical machine translation. In *Proceedings of the Joint Fifth Workshop on Statistical Machine Translation and MetricsMATR*, pages 418–427, Uppsala, Sweden, July 2010. Association for Computational Linguistics. URL https://aclanthology.org/W10-1762.

[89] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. In *Advances in neural information processing systems*, pages 3104–3112, 2014.

[90] Xu Tan, Jiale Chen, Di He, Yingce Xia, Tao Qin, and Tie-Yan Liu. Multilingual neural machine translation with language clustering. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 963–973, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1089. URL https://aclanthology.org/D19-1089.

[91] Pasindu Tennage, Prabath Sandaruwan, Malith Thilakarathne, Achini Herath, and Surangika Ranathunga. Handling rare word problem using synthetic training data for Sinhala and Tamil neural machine translation. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*, Miyazaki, Japan, May 2018. European Language Resources Association (ELRA). URL https://www.aclweb.org/anthology/L18-1261.

[92] Jörg Tiedemann. Parallel data, tools and interfaces in opus. In *Lrec*, volume 2012, pages 2214–2218. Citeseer, 2012.

[93] Zhaopeng Tu, Zhengdong Lu, Yang Liu, Xiaohua Liu, and Hang Li. Modeling coverage for neural machine translation. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 76–85, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1008. URL https://aclanthology.org/P16-1008.

[94] Zhaopeng Tu, Yang Liu, Zhengdong Lu, Xiaohua Liu, and Hang Li. Context gates for neural machine translation. *Transactions of the Association for Computational Linguistics*, 5:87–99, 2017. doi: 10.1162/tacl_a_00048. URL https://aclanthology.org/Q17-1007.

[95] Zhaopeng Tu, Yang Liu, Lifeng Shang, Xiaohua Liu, and Hang Li. Neural machine translation with reconstruction. In *Thirty-First AAAI Conference on Artificial Intelligence*, 2017.

[96] Masao Utiyama and Hitoshi Isahara. A comparison of pivot methods for phrase-based statistical machine translation. In *Human Language Technologies 2007: The Conference of the North American Chapter of the Association for Computational Linguistics; Proceedings of the Main Conference*, pages 484–491, Rochester, New York, April 2007. Association for Computational Linguistics. URL https://aclanthology.org/N07-1061.

[97] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.

[98] Dekai Wu. Stochastic inversion transduction grammars with application to segmentation, bracketing, and alignment of parallel corpora. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence - Volume 2*, IJCAI'95, page 1328–1335, San Francisco, CA, USA, 1995. Morgan Kaufmann Publishers Inc. ISBN 1558603638.

[99] Hua Wu and Haifeng Wang. Pivot language approach for phrase-based statistical machine translation. In *Proceedings of the 45th Annual Meeting of the Association of Computational Linguistics*, pages 856–863, Prague, Czech Republic, June 2007. Association for Computational Linguistics. URL https://www.aclweb.org/anthology/P07-1108.

[100] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Łukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google's neural machine translation system: Bridging the gap between human and machine translation. *CoRR*, abs/1609.08144, 2016. URL http://arxiv.org/abs/1609.08144.

[101] Saumitra Yadav and Manish Shrivastava. A3-108 machine translation system for similar language translation shared task 2020. In *Proceedings of the 5th Conference on Machine Translation (WMT)*, pages 449–453. Association for Computational Linguistics, 19-20 November 2020. URL http://www.statmt.org/wmt20/pdf/2020.wmt-1.55.pdf.

[102] Saumitra Yadav and Manish Shrivastava. A3-108 machine translation system for LoResMT shared task @MT summit 2021 conference. In *Proceedings of the 4th Workshop on Technologies for MT of Low Resource Languages (LoResMT2021)*, pages 124–128, Virtual, August 2021. Association for Machine Translation in the Americas. URL https://aclanthology.org/2021.mtsummit-loresmt.12.

[103] Saumitra Yadav, Vandan Mujadia, and Manish Shrivastava. A3-108 machine translation system for LoResMT 2019. In *Proceedings of the 2nd Workshop on Technologies for MT of Low Resource Languages*, pages 64–67, Dublin, Ireland, 20 August 2019. European Association for Machine Translation. URL https://www.aclweb.org/anthology/W19-6810.

[104] Barret Zoph, Deniz Yuret, Jonathan May, and Kevin Knight. Transfer learning for low-resource neural machine translation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1568–1575, Austin, Texas, November 2016. Association for Computational Linguistics. doi: 10.18653/v1/D16-1163. URL https://www.aclweb.org/anthology/D16-1163.

Chapter 6

List of Selected Papers

Preporcessing Centric Papers

These papers make use of algorithms to preprocess input in different ways which can aid translation in agglutinative language pairs.

S.No.	Paper	Citation
1	Rico Sennrich, Barry Haddow, and Alexandra Birch. "Neural Machine Translation of Rare Words with Subword Units." In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2016.	4334
2	Anoop Kunchukuttan, Abhijit Mishra, Rajen Chatterjee, Ritesh Shah, and Pushpak Bhattacharyya. "Sata-anuvadak: Tackling multiway translation of Indian languages." *pan* 841, no. 54,570 (2014): 4-135.	45
3	Tetsuji Nakagawa. "Efficient top-down BTG parsing for machine translation preordering." In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. 2015.	34
4	Shaohui Kuang and Deyi Xiong. "Automatic long sentence segmentation for neural machine translation." *Natural Language Understanding and Intelligent Applications.* Springer, Cham, 2016. 162-174.	10

Data Augmentation

These paper either directly or indirectly can be used to create synthetic datasets which can be used to further increase performance of translation systems in low resource setting or aid translation systems for domain adaptation.

S.No.	Paper	Citation
5	Rico Sennrich, Barry Haddow, and Alexandra Birch. "Improving Neural Machine Translation Models with Monolingual Data." In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2016.	1516

6	Sergey Edunov, Myle Ott, Michael Auli, and David Grangier. "Understanding back-translation at scale." In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pages 489–500, Brussels, Belgium, October-November 2018.* Association for Computational Linguistics	550
7	Marzieh Fadaee, Arianna Bisazza, and Christof Monz. "Data augmentation for low-resource neural machine translation." In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), 2017.*	282
8	Ahmad Rashid, Alan Do-Omri, Md Haidar, Qun Liu, and Mehdi Rezagholizadeh (2019, June). "Bilingual-GAN: A Step Towards Parallel Text Generation." In *Proceedings of the Workshop on Methods for Optimizing and Evaluating Neural Language Generation (pp. 55-64).*	5
9	Kai Song, Yue Zhang, Heng Yu, Weihua Luo, Kun Wang, and Min Zhang.(2019, June). "Code-Switching for Enhancing NMT with Pre-Specified Translation." In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers) (pp. 449-459).*	42

Learning with Other languages

Paper using Multilingual Translation system with language grouping to learn optimal translation system.

S.No.	**Paper**	**Citation**
10	Xu Tan, Jiale Chen, Di He, Yingce Xia, Tao Qin, and Tie-Yan Liu. "Multilingual Neural Machine Translation with Language Clustering." In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp. 963-973. 2019.*	41